

Laboratorio 3

Sean bienvenidos de nuevo al laboratorio 3 de Deep Learning y Sistemas Inteligentes. Así como en los laboratorios pasados, espero que esta ejercitación les sirva para consolidar sus conocimientos en el tema de Redes Neuronales Recurrentes y LSTM.

Este laboratorio consta de dos partes. En la primera trabajaremos una Red Neuronal Recurrente paso-a-paso. En la segunda fase, usaremos PyTorch para crear una nueva Red Neuronal pero con LSTM, con la finalidad de que no solo sepan que existe cierta función sino también entender qué hace en un poco más de detalle.

Para este laboratorio estaremos usando una herramienta para Jupyter Notebooks que facilitará la calificación, no solo asegurando que ustedes tengan una nota pronto sino también mostrándoles su nota final al terminar el laboratorio.

Espero que esta vez si se muestren los *marks*. De nuevo me discupo si algo no sale bien, seguiremos mejorando conforme vayamos iterando. Siempre pido su comprensión y colaboración si algo no funciona como debería.

Al igual que en el laboratorio pasado, estaremos usando la librería de Dr John Williamson et al de la University of Glasgow, además de ciertas piezas de código de Dr Bjorn Jensen de su curso de Introduction to Data Science and System de la University of Glasgow para la visualización de sus calificaciones.

NOTA: Ahora tambien hay una tercera dependencia que se necesita instalar. Ver la celda de abajo por favor

In [135...

```
# # Una vez instalada la librería por favor, recuerden volverla a comentar.
# !pip install -U --force-reinstall --no-cache https://github.com/johnhw/jhwutils/z
# !pip install scikit-image
# !pip install -U --force-reinstall --no-cache https://github.com/Albert5789/Lautil
```

In [136...

```
import numpy as np
import copy
import matplotlib.pyplot as plt
import scipy
from PIL import Image
import os
from collections import defaultdict

#from IPython import display
#from base64 import b64decode

# Other imports
from unittest.mock import patch
```

```

from uuid import getnode as get_mac

from jhwutils.checkarr import array_hash, check_hash, check_scalar, check_string, a
import jhwutils.image_audio as ia
import jhwutils.tick as tick
from lautils.gradeutils import new_representation, hex_to_float, compare_numbers, c

###
tick.reset_marks()

%matplotlib inline

```

In [137... `# Seeds`
`seed_ = 2023`
`np.random.seed(seed_)`

In [138... `# Celda escondida para utlidades necesarias, por favor NO edite esta celda`

Información del estudiante en dos variables

- carne_1 : un string con su carne (e.g. "12281"), debe ser de al menos 5 caracteres.
- firma_mecanografiada_1: un string con su nombre (e.g. "Albero Suriano") que se usará para la declaracion que este trabajo es propio (es decir, no hay plagio)
- carne_2 : un string con su carne (e.g. "12281"), debe ser de al menos 5 caracteres.
- firma_mecanografiada_2: un string con su nombre (e.g. "Albero Suriano") que se usará para la declaracion que este trabajo es propio (es decir, no hay plagio)

In [139... `carne_1 = "21471"`
`firma_mecanografiada_1 = "Sebastian Juarez"`
`carne_2 = "21471"`
`firma_mecanografiada_2 = "Sebastian Juarez"`
`# YOUR CODE HERE`
`# raise NotImplementedError()`

In [140... `# Deberia poder ver dos checkmarks verdes [0 marks], que indican que su información`
`with tick.marks(0):`
 `assert(len(carne_1)>=5 and len(carne_2)>=5)`
`with tick.marks(0):`
 `assert(len(firma_mecanografiada_1)>0 and len(firma_mecanografiada_2)>0)`

✓ [0 marks]

✓ [0 marks]

Parte 1 - Construyendo una Red Neuronal Recurrente

Créditos: La primera parte de este laboratorio está tomado y basado en uno de los laboratorios dados dentro del curso de "Deep Learning" de Jes Frellsen (DeepLearningDTU)

La aplicación de los datos secuenciales pueden ir desde predicción del clima hasta trabajar con lenguaje natural. En este laboratorio daremos un vistazo a como las RNN pueden ser usadas dentro del modelaje del lenguaje, es decir, trataremos de predecir el siguiente token dada una secuencia. En el campo de NLP, un token puede ser un caracter o bien una palabra.

Representación de Tokens o Texto

Como bien hemos hablado varias veces, la computadora no entiende palabras ni mucho menos oraciones completas en la misma forma que nuestros cerebros lo hacen. Por ello, debemos encontrar alguna forma de representar palabras o caracteres en una manera que la computadora sea capaz de interpretarla, es decir, con números. Hay varias formas de representar un grupo de palabras de forma numérica, pero para fines de este laboratorio vamos a centrarnos en una manera común, llamada "one-hot encoding".

One Hot Encoding

Esta técnica debe resultarles familiar de cursos pasados, donde se tomaba una conjunto de categorías y se les asignaba una columna por categoría, entonces se coloca un 1 si el row que estamos evaluando es parte de esa categoría o un 0 en caso contrario. Este mismo acercamiento podemos tomarlo para representar conjuntos de palabras. Por ejemplo

```
casa = [1, 0, 0, ..., 0]
perro = [0, 1, 0, ..., 0]
```

Representar un vocabulario grande con one-hot encoding, suele volverse ineficiente debido al tamaño de cada vector disperso. Para solventar esto, una práctica común es truncar el vocabulario para contener las palabras más utilizadas y representar el resto con un símbolo especial, UNK, para definir palabras "desconocidas" o "sin importancia". A menudo esto se hace que palabras tales como nombres se vean como UNK porque son raros.

Generando el Dataset a Usar

Para este laboratorio usaremos un dataset simplificado, del cual debería ser más sencillo el aprender de él. Estaremos generando secuencias de la forma

```
a b EOS
a a a a b b b b EOS
```

Noten la aparición del token "EOS", el cual es un caracter especial que denota el fin de la secuencia. Nuestro task en general será el predecir el siguiente token t_n , donde este podrá ser "a", "b", "EOS", o "UNK" dada una secuencia de forma $t_1, \dots, t_{n-1}$.

```
In [141... # Reseed the cell
np.random.seed(seed_)

def generate_data(num_seq=100):
    """
    Genera un grupo de secuencias, la cantidad de secuencias es dada por num_seq

    Args:
    num_seq: El número de secuencias a ser generadas

    Returns:
    Una lista de secuencias
    """
    samples = []
    for i in range(num_seq):
        # Genera una secuencia de largo aleatorio
        num_tokens = np.random.randint(1,12)
        # Genera la muestra
        sample = ['a'] * num_tokens + ['b'] * num_tokens + ['EOS']
        # Agregamos
        samples.append(sample)
    return samples

sequences = generate_data()
print("Una secuencia del grupo generado")
print(sequences[0])
```

Una secuencia del grupo generado

```
['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'EOS']
```

Representación de tokens como índices

En este paso haremos la parte del one-hot encoding. Para esto necesitaremos asignar a cada posible palabra de nuestro vocabulario un índice. Para esto crearemos dos diccionarios, uno que permitirá que dada una palabra nos dirá su representación como "índice" en el vocabulario, y el segundo que irá en dirección contraria.

A estos les llamaremos `word_to_idx` y `idx_to_word`. La variable `vocab_size` nos dirá el máximo de tamaño de nuestro vocabulario. Si intentamos acceder a una palabra que no está en nuestro vocabulario, entonces se le reemplazará con el token "UNK" o su índice correspondiente.

```
In [142... def seqs_to_dicts(sequences):
    """
    Crea word_to_idx y idx_to_word para una lista de secuencias
```

Args:

sequences: lista de secuencias a usar

Returns:

Diccionario de palabra a indice

Diccionario de indice a palabra

Int numero de secuencias

Int tamaño del vocabulario

"""

Lambda para aplanar (flatten) una lista de listas

flatten = lambda l: [item for sublist in l for item in sublist]

Aplanamos el dataset

all_words = flatten(sequences)

Conteo de las ocurrencias de las palabras

word_count = defaultdict(int)

for word in all_words:

word_count[word] += 1

Ordenar por frecuencia

word_count = sorted(list(word_count.items()), key=lambda x: -x[1])

Crear una lista de todas las palabras únicas

unique_words = [w[0] for w in word_count]

Agregamos UNK a la lista de palabras

unique_words.append("UNK")

Conteo del número de secuencias y el número de palabras únicas

num_sentences, vocab_size = len(sequences), len(unique_words)

Crear diccionarios mencionados

word_to_idx = defaultdict(lambda: vocab_size-1)

idx_to_word = defaultdict(lambda: 'UNK')

Llenado de diccionarios

for idx, word in enumerate(unique_words):

Aprox 2 líneas para agregar

word_to_idx[word] = idx

idx_to_word[idx] = word

YOUR CODE HERE

raise NotImplementedError()

return word_to_idx, idx_to_word, num_sentences, vocab_size

word_to_idx, idx_to_word, num_sequences, vocab_size = seqs_to_dicts(sequences)

print(f"Tenemos {num_sequences} secuencias y {len(word_to_idx)} tokens unicos inclu

print(f"El indice de 'b' es {word_to_idx['b']}")

print(f"La palabra con indice 1 es {idx_to_word[1]}")

Tenemos 100 secuencias y 4 tokens unicos incluyendo UNK

El indice de 'b' es 1

La palabra con indice 1 es b

In [143...

```

with tick.marks(3):
    assert(check_scalar(len(word_to_idx), '0xc51b9ba8'))

with tick.marks(2):
    assert(check_scalar(len(idx_to_word), '0xc51b9ba8'))

with tick.marks(5):
    assert(check_string(idx_to_word[0], '0xe8b7be43'))

```

✓ [3 marks]

✓ [2 marks]

✓ [5 marks]

Representación de tokens como índices

Como bien sabemos, necesitamos crear nuestro dataset de forma que el se divida en inputs y targets para cada secuencia y luego particionar esto en training, validation y test (80%, 10%, 10%). Debido a que estamos haciendo prediccion de la siguiente palabra, nuestro target es el input movido (shifted) una palabra.

Vamos a usar PyTorch solo para crear el dataset (como lo hicimos con las imagenes de perritos y gatitos de los laboratorios pasados). Aunque esta vez no haremos el dataloader. Recuerden que siempre es buena idea usar un DataLoader para obtener los datos de una forma eficiente, al ser este un generador/iterador. Además, este nos sirve para obtener la información en batches.

In [144...

```

from torch.utils import data

class Dataset(data.Dataset):
    def __init__(self, inputs, targets):
        self.inputs = inputs
        self.targets = targets

    def __len__(self):
        # Return the size of the dataset
        return len(self.targets)

    def __getitem__(self, index):
        # Retrieve inputs and targets at the given index
        X = self.inputs[index]

```

```

        y = self.targets[index]

        return X, y

def create_datasets(sequences, dataset_class, p_train=0.8, p_val=0.1, p_test=0.1):

    # Definimos el tamaño de las particiones
    num_train = int(len(sequences)*p_train)
    num_val = int(len(sequences)*p_val)
    num_test = int(len(sequences)*p_test)

    # Dividir las secuencias en las particiones
    sequences_train = sequences[:num_train]
    sequences_val = sequences[num_train:num_train+num_val]
    sequences_test = sequences[-num_test:]

    # Funcion interna para obtener los targets de una secuencia
    def get_inputs_targets_from_sequences(sequences):
        # Listas vacias
        inputs, targets = [], []

        # Agregar informacion a las listas, ambas listas tienen L-1 palabras de una
        # pero los targets están movidos a la derecha por uno, para que podamos pr
        for sequence in sequences:
            inputs.append(sequence[:-1])
            targets.append(sequence[1:])

        return inputs, targets

    # Obtener inputs y targes para cada subgrupo
    inputs_train, targets_train = get_inputs_targets_from_sequences(sequences_train)
    inputs_val, targets_val = get_inputs_targets_from_sequences(sequences_val)
    inputs_test, targets_test = get_inputs_targets_from_sequences(sequences_test)

    # Creación de datasets
    training_set = dataset_class(inputs_train, targets_train)
    validation_set = dataset_class(inputs_val, targets_val)
    test_set = dataset_class(inputs_test, targets_test)

    return training_set, validation_set, test_set

training_set, validation_set, test_set = create_datasets(sequences, Dataset)

print(f"Largo del training set {len(training_set)}")
print(f"Largo del validation set {len(validation_set)}")
print(f"Largo del test set {len(test_set)}")

```

Largo del training set 80
 Largo del validation set 10
 Largo del test set 10

One-Hot Encodings

Ahora creamos una función simple para obtener la representación one-hot encoding de dado un índice de una palabra. Noten que el tamaño del one-hot encoding es igual a la del vocabulario. Adicionalmente definamos una función para encodear una secuencia.

```
In [145... def one_hot_encode(idx, vocab_size):
    """
    Encodea una sola palabra dado su índice y el tamaño del vocabulario

    Args:
        idx: índice de la palabra
        vocab_size: tamaño del vocabulario

    Returns
        np.array de lagro "vocab_size"
    """
    # Init array encodeado
    one_hot = np.zeros(vocab_size)

    # Setamos el elemento a uno
    one_hot[idx] = 1.0

    return one_hot

def one_hot_encode_sequence(sequence, vocab_size):
    """
    Encodea una secuencia de palabras dado el tamaño del vocabulario

    Args:
        sentence: una lista de palabras a encodear
        vocab_size: tamaño del vocabulario

    Returns
        np.array 3D de tamaño (numero de palabras, vocab_size, 1)
    """
    # Encodear cada palabra en la secuencia
    encoding = np.array([one_hot_encode(word_to_idx[word], vocab_size) for word in
    sequence])

    # Cambiar de forma para tener (num words, vocab size, 1)
    encoding = encoding.reshape(encoding.shape[0], encoding.shape[1], 1)

    return encoding

test_word = one_hot_encode(word_to_idx['a'], vocab_size)
print(f"Encodeado de 'a' con forma {test_word.shape}")

test_sentence = one_hot_encode_sequence(['a', 'b'], vocab_size)
print(f"Encodeado de la secuencia 'a b' con forma {test_sentence.shape}.")
```

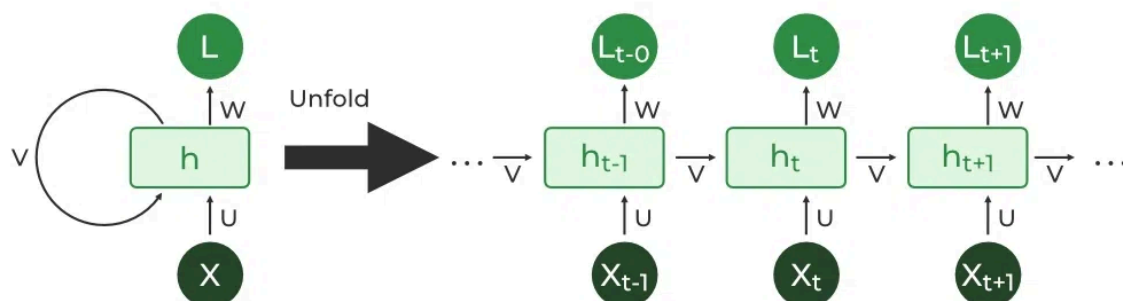
Encodeado de 'a' con forma (4,)

Encodeado de la secuencia 'a b' con forma (2, 4, 1).

Ahora que ya tenemos lo necesario de data para empezar a trabajar, demos paso a hablar un poco más de las RNN

Redes Neuronales Recurrentes (RNN)

Una red neuronal recurrente (RNN) es una red neuronal conocida por modelar de manera efectiva datos secuenciales como el lenguaje, el habla y las secuencias de proteínas. Procesa datos de manera cíclica, aplicando los mismos cálculos a cada elemento de una secuencia. Este enfoque cíclico permite que la red utilice cálculos anteriores como una forma de memoria, lo que ayuda a hacer predicciones para cálculos futuros. Para comprender mejor este concepto, consideren la siguiente imagen.



Crédito de imagen al autor, imagen tomada de "Introduction to Recurrent Neural Network" de Aishwarya.27

Donde:

- x es la secuencia de input
- U es una matriz de pesos aplicada a una muestra de input dada
- V es una matriz de pesos usada para la computación recurrente para pasar la memoria en las secuencias
- W es una matriz de pesos usada para calcular la salida de cada paso
- h es el estado oculto (hidden state) (memoria de la red) para cada paso
- L es la salida resultante

Cuando una red es extendida como se muestra, es más fácil referirse a un paso t . Tenemos los siguientes cálculos en la red

- $h_t = f(Ux_t + Vh_{t-1})$ donde f es la función de activación
- $L_t = \text{softmax}(Wh_t)$

Implementando una RNN

Ahora pasaremos a inicializar nuestra RNN. Los pesos suelen inicializar de forma aleatoria, pero esta vez lo haremos de forma ortogonal para mejorar el rendimiento de nuestra red, y siguiendo las recomendaciones del paper dado abajo.

Tenga cuidado al definir los elementos que se le piden, debido a que una mala dimensión causará que tenga resultados diferentes y errores al operar.

```
In [146... np.random.seed(seed_)

hidden_size = 50 # Numero de dimensiones en el hidden state
vocab_size = len(word_to_idx) # Tamaño del vocabulario

def init_orthogonal(param):
    """
    Initializes weight parameters orthogonally.
    Inicializa los pesos ortogonalmente

    Esta inicialización está dada por el siguiente paper:
    https://arxiv.org/abs/1312.6120
    """
    if param.ndim < 2:
        raise ValueError("Only parameters with 2 or more dimensions are supported.")

    rows, cols = param.shape

    new_param = np.random.randn(rows, cols)

    if rows < cols:
        new_param = new_param.T

    # Calcular factorización QR
    q, r = np.linalg.qr(new_param)

    # Hacer Q uniforme de acuerdo a https://arxiv.org/pdf/math-ph/0609050.pdf
    d = np.diag(r, 0)
    ph = np.sign(d)
    q *= ph

    if rows < cols:
        q = q.T

    new_param = q

    return new_param

def init_rnn(hidden_size, vocab_size):
    """
    Inicializa la RNN

    Args:
        hidden_size: Dimensiones del hidden state
        vocab_size: Dimensión del vocabulario
    """
```

```

# Aprox 5 lineas para
# Definir La matriz de pesos (input del hidden state)
U = np.zeros((hidden_size, vocab_size))
# Definir La matriz de pesos de los calculos recurrentes
V = np.zeros((hidden_size, hidden_size))
# Definir La matriz de pesos del hidden state a la salida
W = np.zeros((vocab_size, hidden_size))
# Bias del hidden state
b_hidden = np.zeros((hidden_size, 1))
# Bias de la salida
b_out = np.zeros((vocab_size, 1))
# Para estas use np.zeros y asegurese de darle las dimensiones correcta a cada
# YOUR CODE HERE
# raise NotImplementedError()

# Aprox 3 lineas para inicializar los pesos de forma ortogonal usando la
# funcion init_orthogonal
U = init_orthogonal(U)
V = init_orthogonal(V)
W = init_orthogonal(W)
# YOUR CODE HERE
# raise NotImplementedError()

# Return parameters as a tuple
return U, V, W, b_hidden, b_out

```

```
params = init_rnn(hidden_size=hidden_size, vocab_size=vocab_size)
```

In [147...

```

with tick.marks(5):
    assert check_hash(params[0], ((50, 4), 80.24369675632171))

with tick.marks(5):
    assert check_hash(params[1], ((50, 50), 3333.838548574836))

with tick.marks(5):
    assert check_hash(params[2], ((4, 50), -80.6410290517092))

with tick.marks(5):
    assert check_hash(params[3], ((50, 1), 0.0))

with tick.marks(5):
    assert check_hash(params[4], ((4, 1), 0.0))

```

✓ [5 marks]

✓ [5 marks]

✓ [5 marks]

✓ [5 marks]

✓ [5 marks]

Funciones de Activación

A continuación definiremos las funciones de activación a usar, sigmoide, tanh y softmax.

In [148...

```
def sigmoid(x, derivative=False):
    """
    Calcula la función sigmoide para un array x

    Args:
        x: El array sobre el que trabajar
        derivative: Si esta como verdadero, regresar el valor en la derivada
    """
    x_safe = x + 1e-12 #Evitar ceros
    # Aprox 1 linea sobre x_safe para implementar La funcion
    f = 1 / (1 + np.exp(-x_safe))
    # YOUR CODE HERE
    # raise NotImplementedError()

    # Regresa La derivada de La funcion
    if derivative:
        return f * (1 - f)
    # Regresa el valor para el paso forward
    else:
        return f

def tanh(x, derivative=False):
    """
    Calcula la función tanh para un array x

    Args:
        x: El array sobre el que trabajar
        derivative: Si esta como verdadero, regresar el valor en la derivada
    """
    x_safe = x + 1e-12 #Evitar ceros
    # Aprox 1 linea sobre x_safe para implementar La funcion
    f = np.tanh(x_safe)
    # YOUR CODE HERE
    # raise NotImplementedError()
```

```

# Regresa La derivada de La funcion
if derivative:
    return 1-f**2
# Regresa el valor para el paso forward
else:
    return f

def softmax(x, derivative=False):
    """
    Calcula la función softmax para un array x

    Args:
    x: El array sobre el que trabajar
    derivative: Si esta como verdadero, regresar el valor en la derivada
    """
    x_safe = x + 1e-12 #Evitar ceros
    # Aprox 1 linea sobre x_safe para implementar La funcion
    f = np.exp(x_safe) / np.sum(np.exp(x_safe), axis=0, keepdims=True)
    # YOUR CODE HERE
    # raise NotImplementedError()

    # Regresa La derivada de La funcion
    if derivative:
        pass # No se necesita en backprog
    # Regresa el valor para el paso forward
    else:
        return f

```

In [149...

```

with tick.marks(5):
    assert check_hash(sigmoid(params[0][0]), ((4,)), 6.997641543410888))

with tick.marks(5):
    assert check_hash(tanh(params[0][0]), ((4,)), -0.007401604025076086))

with tick.marks(5):
    assert check_hash(softmax(params[0][0]), ((4,)), 3.504688021096135))

```

✓ [5 marks]

✓ [5 marks]

✓ [5 marks]

Implementación del paso Forward

Ahora es el momento de implementar el paso forward usando lo que hemos implementado hasta ahora

```
In [150... def forward_pass(inputs, hidden_state, params):
    """
    Calcula el paso forward de RNN

    Args:
        inputs: Seccuencia de input a ser procesada
        hidden_state: Un estado inicializado hidden state
        params: Parametros de la RNN
    """
    # Obtener Los parametros
    U, V, W, b_hidden, b_out = params

    # Crear una lista para guardar las salidas y los hidden states
    outputs, hidden_states = [], []

    # Para cada elemento en la secuencia input
    for t in range(len(inputs)):

        # Aprox 1 line para
        # Calculo del nuevo hidden state usando tanh
        # Recuerden que al ser el hidden state tienen que usar los pesos del input
        # a esto sumarle los pesos recurrentes por el hidden state y finalmente su
        hidden_state = tanh(np.dot(U, inputs[t]) + np.dot(V, hidden_state) + b_hidd
        # YOUR CODE HERE
        # raise NotImplementedError()

        # Aprox 1 linea
        # para el calculo del output
        # Al ser la salida, deben usar softmax sobre la multiplicación de pesos de
        # es decir el calculado en el paso anterior y siempre sumarle su bias con
        out = softmax(
            np.dot(W, hidden_state) + b_out
        )
        # YOUR CODE HERE
        # raise NotImplementedError()

        # Guardamos los resultados y continuamos
        outputs.append(out)
        hidden_states.append(hidden_state.copy())

    return outputs, hidden_states
```

```
In [151... test_input_sequence, test_target_sequence = training_set[0]

# One-hot encode
test_input = one_hot_encode_sequence(test_input_sequence, vocab_size)
test_target = one_hot_encode_sequence(test_target_sequence, vocab_size)

# Init hidden state con zeros
h0 = np.zeros((hidden_size, 1))

outputs, hidden_states = forward_pass(test_input, h0, params)
```

```

print("Secuencia Input:")
print(test_input_sequence)

print("Secuencia Target:")
print(test_target_sequence)

print("Secuencia Predicha:")
print([idx_to_word[np.argmax(output)] for output in outputs])

with tick.marks(5):
    assert check_hash(outputs, ((16, 4, 1), 519.7419046193046))

```

Secuencia Input:

['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b']

Secuencia Target:

['a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'EOS']

Secuencia Predicha:

['a', 'b', 'a', 'a', 'a', 'EOS', 'EOS', 'EOS', 'EOS', 'EOS', 'EOS', 'EOS', 'b', 'b', 'b', 'b']

✓ [5 marks]

Implementación del paso Backward

Ahora es momento de implementar el paso backward. Si se pierden, remítanse a las ecuaciones e imagen dadas previamente.

Usaremos una función auxiliar para evitar la explosión del gradiente. Esta técnica suele funcionar muy bien, si quieren leer más sobre esto pueden consultar estos enlaces

[Understanding Gradient Clipping \(and How It Can Fix Exploding Gradients Problem\)](#)

[What exactly happens in gradient clipping by norm?](#)

In [152...

```

def clip_gradient_norm(grads, max_norm=0.25):
    """
    Clipa (recorta?) el gradiente para tener una norma máxima de `max_norm`
    Esto ayudará a prevenir el problema de la gradiente explosiva (BOOM!)
    """
    # Setea el máximo de la norma para que sea flotante
    max_norm = float(max_norm)
    total_norm = 0

    # Calculamos la norma L2 al cuadrado para cada gradiente y agregamos estas a la
    for grad in grads:
        grad_norm = np.sum(np.power(grad, 2))
        total_norm += grad_norm
    # Cuadrado de la normal total
    total_norm = np.sqrt(total_norm)

    # Calculamos el coeficiente de recorte

```

```

clip_coef = max_norm / (total_norm + 1e-6)

# Si el total de la norma es más grande que el máximo permitido, se recorta la
if clip_coef < 1:
    for grad in grads:
        grad *= clip_coef
return grads

def backward_pass(inputs, outputs, hidden_states, targets, params):
    U, V, W, b_hidden, b_out = params
    d_U, d_V, d_W = np.zeros_like(U), np.zeros_like(V), np.zeros_like(W)
    d_b_hidden, d_b_out = np.zeros_like(b_hidden), np.zeros_like(b_out)
    d_h_next = np.zeros_like(hidden_states[0])
    loss = 0

    for t in reversed(range(len(outputs))):
        loss += -np.sum(targets[t] * np.log(outputs[t] + 1e-12))

        d_o = outputs[t].copy()
        d_o[...] -= targets[t]

        d_W += np.dot(d_o, hidden_states[t].T)
        d_b_out += d_o

        d_h = np.dot(W.T, d_o) + d_h_next
        d_f = (1 - hidden_states[t] ** 2) * d_h
        d_b_hidden += d_f

        d_U += np.dot(d_f, inputs[t].T)
        prev_h = hidden_states[t - 1] if t > 0 else np.zeros_like(hidden_states[0])
        d_V += np.dot(d_f, prev_h.T)
        d_h_next = np.dot(V.T, d_f)

    grads = d_U, d_V, d_W, d_b_hidden, d_b_out
    grads = clip_gradient_norm(grads)

    loss /= len(outputs) # promedio por paso de tiempo
    return loss, grads

```

```

In [153... loss, grads = backward_pass(test_input, outputs, hidden_states, test_target, params

with tick.marks(5):
    assert check_scalar(loss, '0xae59c30')

with tick.marks(5):
    assert check_hash(grads[0], ((50, 4), -16.23575258072454 ))

with tick.marks(5):
    assert check_hash(grads[1], ((50, 50), -47.97665737019572))

with tick.marks(5):
    assert check_hash(grads[2], ((4, 50), 1.629610713756668))

```


✓ [5 marks]

✓ [5 marks]

✓ [5 marks]

✓ [5 marks]

Optimización

Considerando que ya tenemos el paso forward y podemos calcular gradientes con el backpropagation, ya podemos pasar a entrenar nuestra red. Para esto necesitaremos un optimizador. Una forma común y sencilla es implementar la gradiente descendiente.

Recuerden la regla de optimización

$$\theta = \theta - \alpha * \nabla J(\theta)$$

- θ son los parametros del modelo
- α es el learning rate
- $\nabla J(\theta)$ representa la gradiente del costo J con respecto de los parametros

In [154...

```
def update_parameters(params, grads, lr=1e-3):  
    # Iteramos sobre los parametros y las gradientes  
    for param, grad in zip(params, grads):  
        param -= lr * grad  
  
    return params
```

Entrenamiento

Debemos establecer un ciclo de entrenamiento completo que involucre un paso forward, un paso backprop, un paso de optimización y validación. Se espera que el proceso de training dure aproximadamente 5 minutos (o menos), lo que le brinda la oportunidad de continuar leyendo mientras se ejecuta 😊

Noten que estaremos viendo la pérdida en el de validación (no en el de testing) esto se suele hacer para ir observando que tan bien va comportándose el modelo en términos de

generalización. Muchas veces es más recomendable ir viendo como evoluciona la métrica de desempeño principal (accuracy, recall, etc).

```
In [155... # Hyper parametro
# Se coloca como "repsuesta" para que la herramienta no modifique el numero de iter
num_epochs = 2000
# YOUR CODE HERE
# raise NotImplementedError()

# Init una nueva RNN
params = init_rnn(hidden_size=hidden_size, vocab_size=vocab_size)

# Init hiddent state con ceros
hidden_state = np.zeros((hidden_size, 1))

# Rastreo de perdida (loss) para training y validacion
training_loss, validation_loss = [], []

# Iteramos para cada epoca
for i in range(num_epochs):

    # Perdidas en zero
    epoch_training_loss = 0
    epoch_validation_loss = 0

    # Para cada secuencia en el grupo de validación
    for inputs, targets in validation_set:

        # One-hot encode el input y el target
        inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)
        targets_one_hot = one_hot_encode_sequence(targets, vocab_size)

        # Re-init el hidden state
        hidden_state = np.zeros_like(hidden_state)

        # Aprox 1 Line para el paso forward
        outputs, hidden_states = forward_pass(inputs_one_hot, hidden_state, params)
        # YOUR CODE HERE
        # raise NotImplementedError()

        # Aprox 1 Line para el paso backward
        loss, _ = backward_pass(inputs_one_hot, outputs, hidden_states, targets_one_hot)
        # YOUR CODE HERE
        # raise NotImplementedError()

        # Actualización de perdida
        epoch_validation_loss += loss

    # For each sentence in training set
    for inputs, targets in training_set:

        # One-hot encode el input y el target
        inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)
        targets_one_hot = one_hot_encode_sequence(targets, vocab_size)
```

```

# Re-init el hidden state
hidden_state = np.zeros_like(hidden_state)

# Aprox 1 line para el paso forward
outputs, hidden_states = forward_pass(inputs_one_hot, hidden_state, params)
# YOUR CODE HERE
# raise NotImplementedError()

# Aprox 1 line para el paso backward
loss, grads = backward_pass(inputs_one_hot, outputs, hidden_states, targets)
# YOUR CODE HERE
# raise NotImplementedError()

# Validar si la perdida es nan, llegamos al problema del vanishing gradient
if np.isnan(loss):
    raise ValueError("La gradiente se desvanecio... POOF!")

# Actualización de parámetros
params = update_parameters(params, grads, lr=3e-4)

# Actualización de perdida
epoch_training_loss += loss

# Guardar la perdida para graficar
training_loss.append(epoch_training_loss/len(training_set))
validation_loss.append(epoch_validation_loss/len(validation_set))

# Mostrar la perdida cada 100 epocas
if i % 100 == 0:
    print(f'Epoca {i}, training loss: {training_loss[-1]}, validation loss: {va

```

```

Epoca 0, training loss: 1.4282933933208766, validation loss: 1.4371127143796578
Epoca 100, training loss: 1.027359889082519, validation loss: 1.0026687442611681
Epoca 200, training loss: 0.8455108538566675, validation loss: 0.8042919332118019
Epoca 300, training loss: 0.7651632001046991, validation loss: 0.7196934804961466
Epoca 400, training loss: 0.7234374792382565, validation loss: 0.6773094175507864
Epoca 500, training loss: 0.6946998230748654, validation loss: 0.6484998106947192
Epoca 600, training loss: 0.669690686302907, validation loss: 0.6232837324312677
Epoca 700, training loss: 0.645113815754471, validation loss: 0.5983188816987156
Epoca 800, training loss: 0.619908453905092, validation loss: 0.5726152511019784
Epoca 900, training loss: 0.5943834082820184, validation loss: 0.5467397704506763
Epoca 1000, training loss: 0.5702016129762297, validation loss: 0.5233684774290934
Epoca 1100, training loss: 0.5492031008431881, validation loss: 0.5056959175807284
Epoca 1200, training loss: 0.5317408181707839, validation loss: 0.49398450001664135
Epoca 1300, training loss: 0.5180904303928948, validation loss: 0.4875552149606783
Epoca 1400, training loss: 0.5074080128334011, validation loss: 0.4837979006574633
Epoca 1500, training loss: 0.49822737624407143, validation loss: 0.48017924590423566
Epoca 1600, training loss: 0.4900447999545802, validation loss: 0.47637420674475434
Epoca 1700, training loss: 0.4826112178931713, validation loss: 0.4723027476293956
Epoca 1800, training loss: 0.47571422276778136, validation loss: 0.46779227840013266
Epoca 1900, training loss: 0.4691796060308004, validation loss: 0.4626706074385762

```

In [156...

```

# Veamos la primera secuencia en el test set
inputs, targets = test_set[1]

# One-hot encode el input y el target

```

```

inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)
targets_one_hot = one_hot_encode_sequence(targets, vocab_size)

# Init el hidden state con ceros
hidden_state = np.zeros((hidden_size, 1))

# Hacemos el pase forward para evaluar nuestra secuencia
outputs, hidden_states = forward_pass(inputs_one_hot, hidden_state, params)
output_sentence = [idx_to_word[np.argmax(output)] for output in outputs]
print("Secuencia Input:")
print(inputs)

print("Secuencia Target:")
print(targets)

print("Secuencia Predicha:")
print([idx_to_word[np.argmax(output)] for output in outputs])

# Graficamos la perdida
epoch = np.arange(len(training_loss))
plt.figure()
plt.plot(epoch, training_loss, 'r', label='Training loss',)
plt.plot(epoch, validation_loss, 'b', label='Validation loss')
plt.legend()
plt.xlabel('Epoch'), plt.ylabel('NLL')
plt.show()

with tick.marks(10):
    assert compare_lists_by_percentage(targets, [idx_to_word[np.argmax(output)] for

```

Secuencia Input:

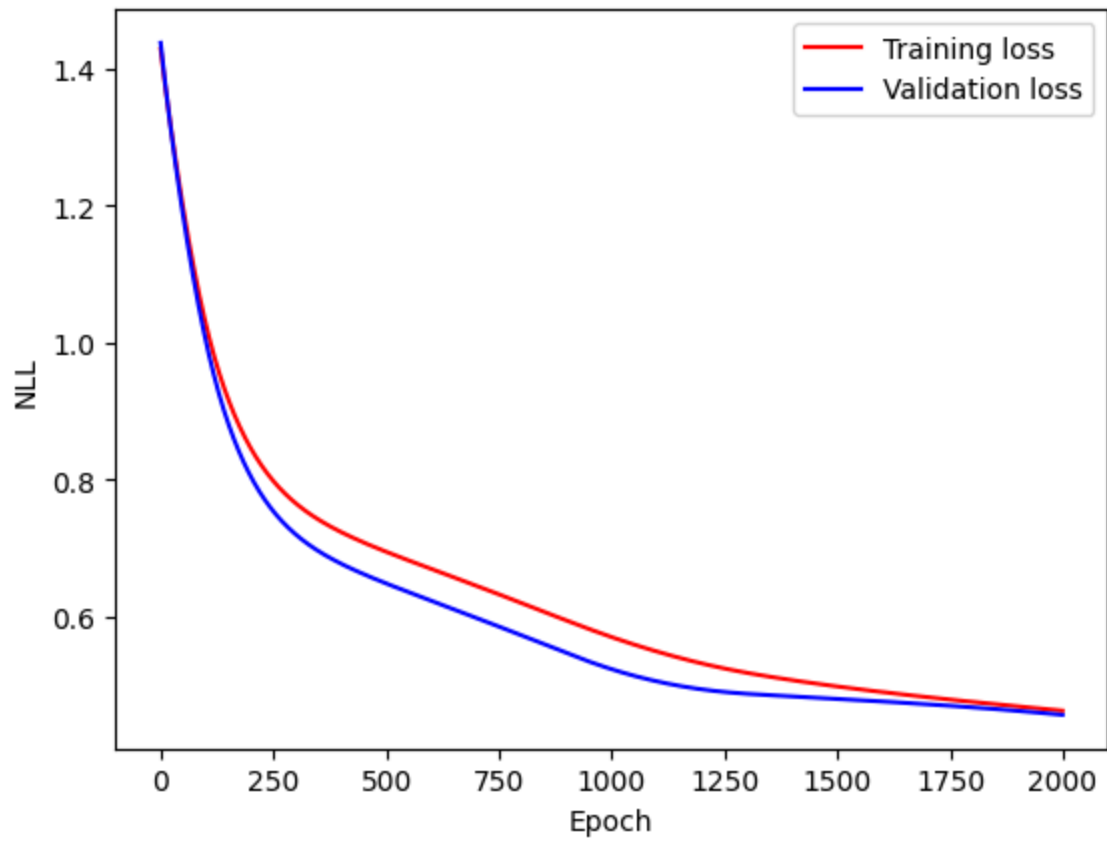
```
['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b',
'b', 'b', 'b', 'b', 'b', 'b']
```

Secuencia Target:

```
['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b',
'b', 'b', 'b', 'b', 'b', 'b', 'EOS']
```

Secuencia Predicha:

```
['a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b',
'b', 'b', 'b', 'EOS', 'EOS', 'EOS']
```



✓ [10 marks]

Preguntas

Ya hemos visto el funcionamiento general de nuestra red RNN, viendo las gráficas de arriba, **responda** lo siguiente dentro de esta celda

- ¿Qué interpretación le da a la separación de las graficas de training y validation?

Podemos ver que el modelo generaliza bien al inicio ya que van bajando juntas las dos lineas, luego aparece una separacion que puede pasar por que no se esta sobreajustando el modelo o hay algun tipo de ruido en medio del entrenamiento.

- ¿Cree que es un buen modelo basado solamente en el loss?

Si solo vemos el Loss podemos pensar que si es un buen modelo. Es por que no se ve que haya overfitting y el loss final tiene valores bastante bajos.

- ¿Cómo deberían de verse esas gráficas en un modelo ideal?

Se deberian ver lineas de training y de validation bastante juntas, estas tienen que bajar rapidamente para luego suavizarse y que no se separen al final. Nuestra grafica solo cumple con las ultimas 2

Parte 2 - Construyendo una Red Neuronal LSTM

Créditos: La segunda parte de este laboratorio está tomado y basado en uno de los laboratorios dados dentro del curso de "Deep Learning" de Jes Frellsen (DeepLearningDTU)

Consideren leer el siguiente blog para mejorar el entendimiento de este tema:

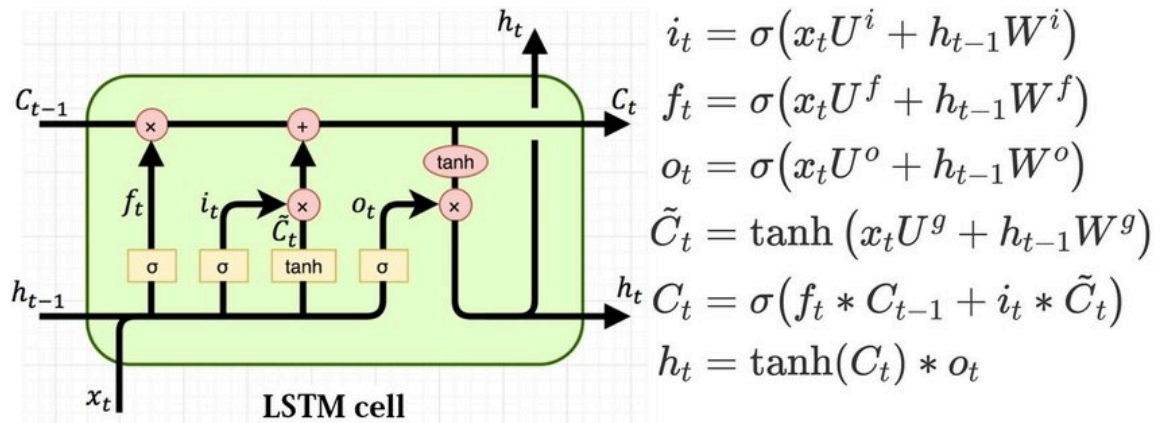
<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

La RNN estándar enfrenta un problema de gradientes que desaparecen, lo que dificulta la retención de memoria en secuencias más largas. Para hacer frente a estos desafíos, se introdujeron algunas variantes.

Los dos tipos principales son la celda de memoria a corto plazo (LSTM) y la unidad recurrente cerrada (GRU), las cuales demuestran una capacidad mejorada para conservar y utilizar la memoria en pasos de tiempo posteriores.

En este ejercicio, nuestro enfoque estará en LSTM, pero los principios aprendidos aquí también se pueden aplicar fácilmente para implementar GRU.

Recordemos una de las imagenes que vimos en clase



Crédito de imagen al autor, imagen tomada de "Designing neural network based decoders for surface codes" de Savvas Varsamopoulos

Recordemos que la "celula" de LST contiene tres tipos de gates, input, forget y output gate. La salida de una unidad LSTM está calculada por las siguientes funciones, donde $\sigma = \text{softmax}$. Entonces tenemos la input gate i , la forget gate f y la output gate o

- $i = \sigma(W^i[h_{t-1}, x_t])$
- $f = \sigma(W^f[h_{t-1}, x_t])$
- $o = \sigma(W^o[h_{t-1}, x_t])$

Donde W^i , W^f , W^o son las matrices de pesos aplicada a cada aplicadas a una matriz contatenada h_{t-1} (hidden state vector) y x_t (input vector) para cada respectiva gate h_{t-1} , del paso previo junto con el input actual x_t son usados para calcular una memoria candidata g

- $g = \tanh(W^g[h_{t-1}, x_t])$

El valor de la memoria c_t es actualizada como

$$c_t = c_{t-1} \circ f + g \circ i$$

donde c_{t-1} es la memoria previa, y $\circ$ es una multiplicacion element-wise (recuerden que este tipo de multiplicación en numpy es con *)

La salida h_t es calculada como

$$h_t = \tanh(c_t) \circ o$$

y este se usa para tanto la salida del paso como para el siguiente paso, mientras c_t es exclusivamente enviado al siguiente paso. Esto hace c_t una memoria feature, y no es usado directamente para caluclar la salida del paso actual.

Iniciando una Red LSTM

De forma similar a lo que hemos hecho antes, necesitaremos implementar el paso forward, backward y un ciclo de entrenamiento. Pero ahora usaremos LSTM con NumPy. Más adelante veremos como es que esto funciona con PyTorch.

In [157...

```

np.random.seed(seed_)

# Tamaño del hidden state concatenado más el input
z_size = hidden_size + vocab_size

def init_lstm(hidden_size, vocab_size, z_size):
    """
    Initializes our LSTM network.
    Init LSTM

    Args:
        hidden_size: Dimensiones del hidden state
        vocab_size: Dimensiones de nuestro vocabulario
        z_size: Dimensiones del input concatenado
    """

    # Aprox 1 linea para empezar la matriz de pesos de la forget gate
    # Recuerden que esta debe empezar con numeros aleatorios
    W_f = np.random.randn(hidden_size, z_size)
    # YOUR CODE HERE
    # raise NotImplementedError()

    # Bias del forget gate
    b_f = np.zeros((hidden_size, 1))

    # Aprox 1 linea para empezar la matriz de pesos de la input gate
    # Recuerden que esta debe empezar con numeros aleatorios
    W_i = np.random.randn(hidden_size, z_size)
    # YOUR CODE HERE
    # raise NotImplementedError()

    # Bias para input gate
    b_i = np.zeros((hidden_size, 1))

    # Aprox 1 linea para empezar la matriz de pesos para la memoria candidata
    # Recuerden que esta debe empezar con numeros aleatorios
    W_g = np.random.randn(hidden_size, z_size)
    # YOUR CODE HERE
    # raise NotImplementedError()

    # Bias para la memoria candidata
    b_g = np.zeros((hidden_size, 1))

    # Aprox 1 linea para empezar la matriz de pesos para la output gate
    # YOUR CODE HERE
    W_o = np.random.randn(hidden_size, z_size)
    # raise NotImplementedError()

    # Bias para la output gate
    b_o = np.zeros((hidden_size, 1))

```



```

# Aprox 1 linea para empezar la matriz que relaciona el hidden state con el out
W_v = np.random.randn(vocab_size, hidden_size)
# YOUR CODE HERE
# raise NotImplementedError()

# Bias
b_v = np.zeros((vocab_size, 1))

# Init pesos ortogonalmente (https://arxiv.org/abs/1312.6120)
W_f = init_orthogonal(W_f)
W_i = init_orthogonal(W_i)
W_g = init_orthogonal(W_g)
W_o = init_orthogonal(W_o)
W_v = init_orthogonal(W_v)

return W_f, W_i, W_g, W_o, W_v, b_f, b_i, b_g, b_o, b_v

params = init_lstm(hidden_size=hidden_size, vocab_size=vocab_size, z_size=z_size)

```

In [158...

```

with tick.marks(5):
    assert check_hash(params[0], ((50, 54), -28071.583543573637))

with tick.marks(5):
    assert check_hash(params[1], ((50, 54), -6337.520066952928))

with tick.marks(5):
    assert check_hash(params[2], ((50, 54), -13445.986473992281))

with tick.marks(5):
    assert check_hash(params[3], ((50, 54), 2276.1116210911564))

with tick.marks(5):
    assert check_hash(params[4], ((4, 50), -201.28961326044097))

```

✓ [5 marks]

✓ [5 marks]

✓ [5 marks]

✓ [5 marks]

✓ [5 marks]

Forward

Vamos para adelante con LSTM, al igual que previamente necesitamos implementar las funciones antes mencionadas

```
In [159... def forward(inputs, h_prev, C_prev, p):
    """
    Arguments:
    x: Input data en el paso "t", shape (n_x, m)
    h_prev: Hidden state en el paso "t-1", shape (n_a, m)
    C_prev: Memoria en el paso "t-1", shape (n_a, m)
    p: Lista con pesos y biases, contiene:
        W_f: Pesos de la forget gate, shape (n_a, n_a + n_x)
        b_f: Bias de la forget gate, shape (n_a, 1)
        W_i: Pesos de la update gate, shape (n_a, n_a + n_x)
        b_i: Bias de la update gate, shape (n_a, 1)
        W_g: Pesos de la primer "tanh", shape (n_a, n_a + n_x)
        b_g: Bias de la primer "tanh", shape (n_a, 1)
        W_o: Pesos de la output gate, shape (n_a, n_a + n_x)
        b_o: Bias de la output gate, shape (n_a, 1)
        W_v: Pesos de la matriz que relaciona el hidden state con e
        b_v: Bias que relaciona el hidden state con el output, shap

    Returns:
    z_s, f_s, i_s, g_s, C_s, o_s, h_s, v_s: Lista de tamaño m conteniendo los calculo
    outputs: Predicciones en el paso "t", shape (n_v, m)
    """

    # Validar las dimensiones
    assert h_prev.shape == (hidden_size, 1)
    assert C_prev.shape == (hidden_size, 1)

    # Desempacar los parametros
    W_f, W_i, W_g, W_o, W_v, b_f, b_i, b_g, b_o, b_v = p

    # Listas para calculos de cada componente en LSTM
    x_s, z_s, f_s, i_s, = [], [], [], []
    g_s, C_s, o_s, h_s = [], [], [], []
    v_s, output_s = [], []

    # Agregar Los valores iniciales
    h_s.append(h_prev)
    C_s.append(C_prev)

    for x in inputs:

        # Aprox 1 linea para concatenar el input y el hidden state
        z = np.row_stack((h_prev, x))
        # YOUR CODE HERE
```

```

# raise NotImplementedError()
z_s.append(z)

# Aprox 1 linea para calcular el forget gate
# Hint: recuerde usar sigmoid
f = sigmoid(np.dot(W_f, z) + b_f)
# YOUR CODE HERE
# raise NotImplementedError()
f_s.append(f)

# Calculo del input gate
i = sigmoid(np.dot(W_i, z) + b_i)
i_s.append(i)

# Calculo de la memoria candidata
g = tanh(np.dot(W_g, z) + b_g)
g_s.append(g)

# Aprox 1 linea para calcular el estado de la memoria
C_prev = f * C_prev + i * g
# YOUR CODE HERE
# raise NotImplementedError()
C_s.append(C_prev)

# Aprox 1 linea para el calculo de la output gate
# Hint: recuerde usar sigmoid
o = sigmoid(np.dot(W_o, z) + b_o)
# YOUR CODE HERE
# raise NotImplementedError()
o_s.append(o)

# Calculate hidden state
# Aprox 1 linea para el calculo del hidden state
h_prev = o * tanh(C_prev)
# YOUR CODE HERE
# raise NotImplementedError()
h_s.append(h_prev)

# Calcular logits
v = np.dot(W_v, h_prev) + b_v
v_s.append(v)

# Calculo de output (con softmax)
output = softmax(v)
output_s.append(output)

return z_s, f_s, i_s, g_s, C_s, o_s, h_s, v_s, output_s

```

```

In [160... # Obtener la primera secuencia para probar
inputs, targets = test_set[1]

# One-hot encode del input y target
inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)
targets_one_hot = one_hot_encode_sequence(targets, vocab_size)

# Init hidden state con ceros

```

```

h = np.zeros((hidden_size, 1))
c = np.zeros((hidden_size, 1))

# Forward
z_s, f_s, i_s, g_s, C_s, o_s, h_s, v_s, outputs = forward(inputs_one_hot, h, c, par

output_sentence = [idx_to_word[np.argmax(output)] for output in outputs]

print("Secuencia Input:")
print(inputs)

print("Secuencia Target:")
print(targets)

print("Secuencia Predicha:")
print([idx_to_word[np.argmax(output)] for output in outputs])

with tick.marks(5):
    assert check_hash(outputs, ((22, 4, 1), 980.1651308051631))

```

Secuencia Input:

['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b']

Secuencia Target:

['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'EOS']

Secuencia Predicha:

['b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'EOS', 'EOS', 'EOS', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b']

✓ [5 marks]

Backward

Ahora de reversa, al igual que lo hecho antes, necesitamos implementar el paso de backward

In [161...

```

def backward(z, f, i, g, C, o, h, v, outputs, targets, p = params):
    """
    Arguments:
    z: Input concatenado como una lista de tamaño m.
    f: Calculos del forget gate como una lista de tamaño m.
    i: Calculos del input gate como una lista de tamaño m.
    g: Calculos de la memoria candidata como una lista de tamaño m.
    C: Celdas estado como una lista de tamaño m+1.
    o: Calculos del output gate como una lista de tamaño m.
    h: Calculos del Hidden State como una lista de tamaño m+1.
    v: Calculos del logit como una lista de tamaño m.
    outputs: Salidas como una lista de tamaño m.
    targets: Targets como una lista de tamaño m.
    p: Lista con pesos y biases, contiene:
        W_f: Pesos de la forget gate, shape (n_a, n_a + n_x)

```

```

        b_f: Bias de la forget gate, shape (n_a, 1)
        W_i: Pesos de la update gate, shape (n_a, n_a + n_x)
        b_i: Bias de la update gate, shape (n_a, 1)
        W_g: Pesos de la primer "tanh", shape (n_a, n_a + n_x)
        b_g: Bias de la primer "tanh", shape (n_a, 1)
        W_o: Pesos de la output gate, shape (n_a, n_a + n_x)
        b_o: Bias de la output gate, shape (n_a, 1)
        W_v: Pesos de la matriz que relaciona el hidden state con e
        b_v: Bias que relaciona el hidden state con el output, shap

Returns:
loss: crossentropy loss para todos los elementos del output
grads: lista de gradientes para todos los elementos en p
"""

# Desempacar parametros
W_f, W_i, W_g, W_o, W_v, b_f, b_i, b_g, b_o, b_v = p

# Init gradientes con cero
W_f_d = np.zeros_like(W_f)
b_f_d = np.zeros_like(b_f)

W_i_d = np.zeros_like(W_i)
b_i_d = np.zeros_like(b_i)

W_g_d = np.zeros_like(W_g)
b_g_d = np.zeros_like(b_g)

W_o_d = np.zeros_like(W_o)
b_o_d = np.zeros_like(b_o)

W_v_d = np.zeros_like(W_v)
b_v_d = np.zeros_like(b_v)

# Setear la proxima unidad y hidden state con ceros
dh_next = np.zeros_like(h[0])
dC_next = np.zeros_like(C[0])

# Para la perdida
loss = 0

# Iteramos en reversa los outputs
for t in reversed(range(len(outputs))):

    # Aprox 1 linea para calcular la perdida con cross entropy
    loss += -np.sum(targets[t] * np.log(outputs[t] + 1e-12))
    # YOUR CODE HERE
    # raise NotImplementedError()

    # Obtener el hidden state del estado previo
    C_prev = C[t-1]

    # Compute the derivative of the relation of the hidden-state to the output
    # Calculo de las derivadas en relacion del hidden state al output gate
    dv = np.copy(outputs[t])
    dv[np.argmax(targets[t])] -= 1

```

```

# Aprox 1 linea para actualizar la gradiente de la relacion del hidden-stat
W_v_d += np.dot(dv, h[t].T)
# YOUR CODE HERE
# raise NotImplementedError()
b_v_d += dv

# Calculo de la derivada del hidden state y el output gate
dh = np.dot(W_v.T, dv)
dh += dh_next
do = dh * tanh(C[t])
# Aprox 1 linea para calcular la derivada del output
do = sigmoid(o[t], True) * do
# Hint: Recuerde multiplicar por el valor previo de do (el de arriba)
# YOUR CODE HERE
# raise NotImplementedError()

# Actualizacion de las gradientes con respecto al output gate
W_o_d += np.dot(do, z[t].T)
b_o_d += do

# Calculo de las derivadas del estado y la memoria candidata g
dC = np.copy(dC_next)
dC += dh * o[t] * tanh(tanh(C[t]), derivative=True)
dg = dC * i[t]
# Aprox 1 linea de codigo para terminar el calculo de dg
dg = tanh(g[t], True) * dg
# YOUR CODE HERE
# raise NotImplementedError()

# Actualización de las gradientes con respecto de la mem candidata
W_g_d += np.dot(dg, z[t].T)
b_g_d += dg

# Compute the derivative of the input gate and update its gradients
# Calculo de la derivada del input gate y la actualización de sus gradientes
di = dC * g[t]
di = sigmoid(i[t], True) * di
# Aprox 2 lineas para el calculo de los pesos y bias del input gate
W_i_d += np.dot(di, z[t].T)
b_i_d += di
# YOUR CODE HERE
# raise NotImplementedError()

# Calculo de las derivadas del forget gate y actualización de sus gradientes
df = dC * C_prev
df = sigmoid(f[t]) * df
# Aprox 2 lineas para el calculo de los pesos y bias de la forget gate
W_f_d += np.dot(df, z[t].T)
b_f_d += df
# YOUR CODE HERE
# raise NotImplementedError()

# Calculo de las derivadas del input y la actualización de gradientes del h
dz = (np.dot(W_f.T, df)
      + np.dot(W_i.T, di)
      + np.dot(W_g.T, dg))

```

```

        + np.dot(W_o.T, do))
    dh_prev = dz[:hidden_size, :]
    dC_prev = f[t] * dC

    grads= W_f_d, W_i_d, W_g_d, W_o_d, W_v_d, b_f_d, b_i_d, b_g_d, b_o_d, b_v_d

    # Recorte de gradientes
    grads = clip_gradient_norm(grads)

    return loss, grads

```

Training

Ahora intentemos entrenar nuestro LSTM básico. Esta parte es muy similar a lo que ya hicimos previamente con la RNN

In [162...

```

# Hyper parametros
num_epochs = 500

# Init una nueva red
z_size = hidden_size + vocab_size # Tamaño del hidden concatenado + el input
params = init_lstm(hidden_size=hidden_size, vocab_size=vocab_size, z_size=z_size)

# Init hidden state como ceros
hidden_state = np.zeros((hidden_size, 1))

# Perdida
training_loss, validation_loss = [], []

# Iteramos cada epoca
for i in range(num_epochs):

    # Perdidas
    epoch_training_loss = 0
    epoch_validation_loss = 0

    # Para cada secuencia en el validation set
    for inputs, targets in validation_set:

        # One-hot encode el inpyt y el target
        inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)
        targets_one_hot = one_hot_encode_sequence(targets, vocab_size)

        # Init hidden state y la unidad de estado como ceros
        h = np.zeros((hidden_size, 1))
        c = np.zeros((hidden_size, 1))

        # Forward
        z_s, f_s, i_s, g_s, C_s, o_s, h_s, v_s, outputs = forward(inputs_one_hot, h

        # Backward
        loss, _ = backward(z_s, f_s, i_s, g_s, C_s, o_s, h_s, v_s, outputs, targets

    # Actualizacion de la perdida

```

```

epoch_validation_loss += loss

# Para cada secuencia en el training set
for inputs, targets in training_set:

    # One-hot encode el input y el target
    inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)
    targets_one_hot = one_hot_encode_sequence(targets, vocab_size)

    # Init hidden state y la unidad de estado como ceros
    h = np.zeros((hidden_size, 1))
    c = np.zeros((hidden_size, 1))

    # Forward
    z_s, f_s, i_s, g_s, C_s, o_s, h_s, v_s, outputs = forward(inputs_one_hot, h

    # Backward
    loss, grads = backward(z_s, f_s, i_s, g_s, C_s, o_s, h_s, v_s, outputs, tar

    # Actualización de parametros
    params = update_parameters(params, grads, lr=1e-1)

    # Actualización de la pérdida
    epoch_training_loss += loss

# Guardar la pérdida para ser graficada
training_loss.append(epoch_training_loss/len(training_set))
validation_loss.append(epoch_validation_loss/len(validation_set))

# Mostrar la pérdida cada 5 épocas
if i % 10 == 0:
    print(f'Epoch {i}, training loss: {training_loss[-1]}, validation loss: {va

```



```

Epoch 0, training loss: 11.954226286622177, validation loss: 17.99882824463402
Epoch 10, training loss: 4.8683982548771585, validation loss: 5.7952856915155975
Epoch 20, training loss: 3.629457778859936, validation loss: 4.3260853126791226
Epoch 30, training loss: 3.7215002044763983, validation loss: 6.363798720536839
Epoch 40, training loss: 3.674832934747762, validation loss: 6.476318408107311
Epoch 50, training loss: 3.535423440643523, validation loss: 5.996159874321436
Epoch 60, training loss: 3.37222680338783, validation loss: 5.4436676943566455
Epoch 70, training loss: 3.2201489206105323, validation loss: 4.895665013369451
Epoch 80, training loss: 3.123677337437442, validation loss: 4.498421900687108
Epoch 90, training loss: 3.0401321751044406, validation loss: 4.210712311148287
Epoch 100, training loss: 2.9648485182950526, validation loss: 4.031675743059035
Epoch 110, training loss: 2.901626956918287, validation loss: 3.8551370832947085
Epoch 120, training loss: 2.8808122330416674, validation loss: 3.8075327539686312
Epoch 130, training loss: 2.877758871006993, validation loss: 3.8271820275163675
Epoch 140, training loss: 2.861981298842099, validation loss: 3.7978994496340706
Epoch 150, training loss: 2.8354793996351955, validation loss: 3.7094687197226692
Epoch 160, training loss: 2.823701353109281, validation loss: 3.6595566633027317
Epoch 170, training loss: 2.8211698025263616, validation loss: 3.654298986525304
Epoch 180, training loss: 2.7941606939549453, validation loss: 3.567300921590202
Epoch 190, training loss: 2.776726538045539, validation loss: 3.503234363517811
Epoch 200, training loss: 2.7770738877565395, validation loss: 3.516604848247632
Epoch 210, training loss: 2.7896210129667542, validation loss: 3.589332538841814
Epoch 220, training loss: 2.8251783952984044, validation loss: 3.753703185813263
Epoch 230, training loss: 2.886424373909442, validation loss: 3.999853444325915
Epoch 240, training loss: 2.9492020319392145, validation loss: 4.241022233762506
Epoch 250, training loss: 2.998973036015482, validation loss: 4.4304239113834285
Epoch 260, training loss: 3.0339014594040608, validation loss: 4.562248001956417
Epoch 270, training loss: 3.0522420425889303, validation loss: 4.631968693360822
Epoch 280, training loss: 3.051218349758547, validation loss: 4.6328651928009235
Epoch 290, training loss: 3.0297088075960787, validation loss: 4.562966327792038
Epoch 300, training loss: 2.9920851376470106, validation loss: 4.435574477222128
Epoch 310, training loss: 2.9518574681764953, validation loss: 4.291863980336198
Epoch 320, training loss: 2.930148426964666, validation loss: 4.200867088285618
Epoch 330, training loss: 2.943408089852767, validation loss: 4.217325491764865
Epoch 340, training loss: 2.998070707706481, validation loss: 4.3532823654868995
Epoch 350, training loss: 3.0962763792442716, validation loss: 4.604228021404
Epoch 360, training loss: 3.177549019667239, validation loss: 4.8077418917072805
Epoch 370, training loss: 3.164133670723502, validation loss: 4.74652717478926
Epoch 380, training loss: 3.067023126435178, validation loss: 4.480020186360536
Epoch 390, training loss: 2.8969847838799, validation loss: 3.9914373796404545
Epoch 400, training loss: 2.797858699210502, validation loss: 3.6363853858372743
Epoch 410, training loss: 2.821340100010851, validation loss: 3.700524806912888
Epoch 420, training loss: 2.8548708091695865, validation loss: 3.818796365580596
Epoch 430, training loss: 2.8699186539767885, validation loss: 3.8720323579011646
Epoch 440, training loss: 2.87487265922458, validation loss: 3.890166509018601
Epoch 450, training loss: 2.8728102201411234, validation loss: 3.8837264079826723
Epoch 460, training loss: 2.8657952107665676, validation loss: 3.8604308512882595
Epoch 470, training loss: 2.854483535121802, validation loss: 3.8235679109991687
Epoch 480, training loss: 2.837027996074677, validation loss: 3.768036117342782
Epoch 490, training loss: 2.808248963452015, validation loss: 3.6772187785523824

```

```

In [163... # Obtener la primera secuencia del test set
            inputs, targets = test_set[1]

            # One-hot encode el input y el target
            inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)

```

```

targets_one_hot = one_hot_encode_sequence(targets, vocab_size)

# Init hidden state como ceros
h = np.zeros((hidden_size, 1))
c = np.zeros((hidden_size, 1))

# Forward
z_s, f_s, i_s, g_s, C_s, o_s, h_s, v_s, outputs = forward(inputs_one_hot, h, c, par

print("Secuencia Input:")
print(inputs)

print("Secuencia Target:")
print(targets)

print("Secuencia Predicha:")
print([idx_to_word[np.argmax(output)] for output in outputs])

# Graficar la perdida en training y validacion
epoch = np.arange(len(training_loss))
plt.figure()
plt.plot(epoch, training_loss, 'r', label='Training loss',)
plt.plot(epoch, validation_loss, 'b', label='Validation loss')
plt.legend()
plt.xlabel('Epoch'), plt.ylabel('NLL')
plt.show()

```

Secuencia Input:

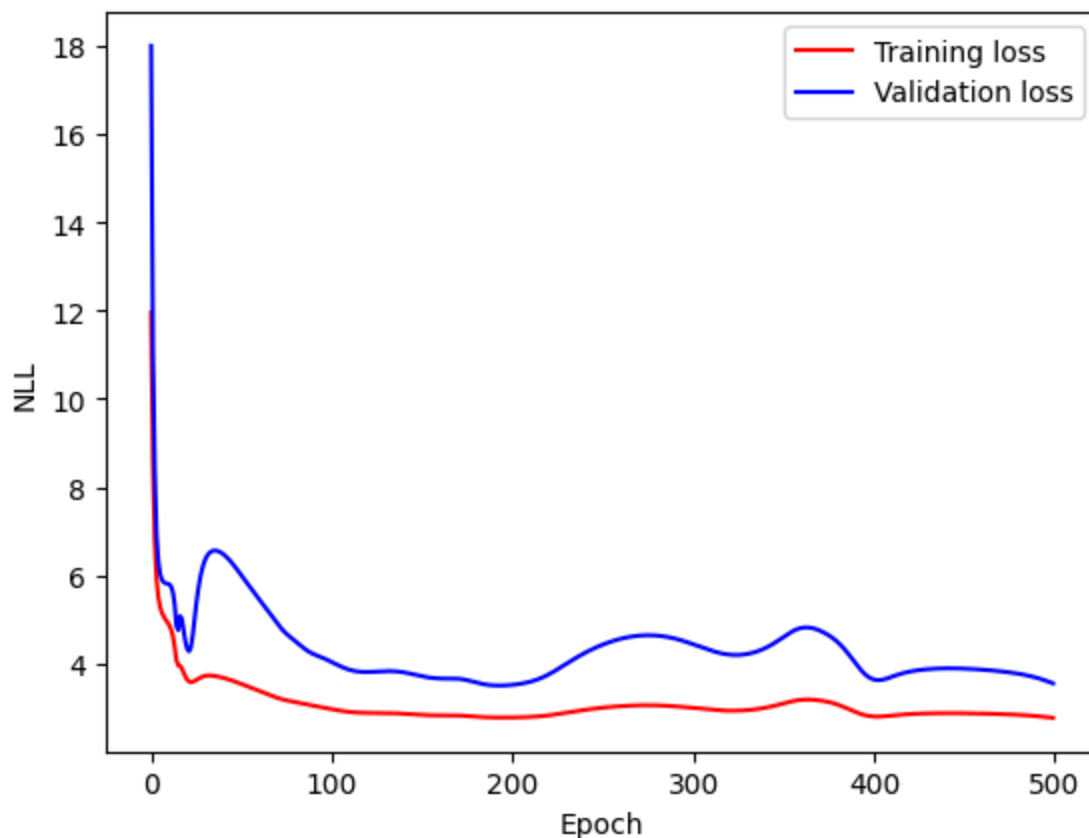
```
['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b',
'b', 'b', 'b', 'b', 'b', 'b']
```

Secuencia Target:

```
['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b',
'b', 'b', 'b', 'b', 'b', 'EOS']
```

Secuencia Predicha:

```
['a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b',
'b', 'b', 'b', 'b', 'b', 'EOS']
```



Preguntas

Respuesta lo siguiente dentro de esta celda

- ¿Qué modelo funcionó mejor? ¿RNN tradicional o el basado en LSTM? ¿Por qué?

El LSTM funciono mejor que la tradicional por que esta fue mas precisa. Vimos que la tradicional fallaba en puntos criticos de la transicion y que la LSTM mantuvo por mas tiempo la memoria del patron

- Observen la gráfica obtenida arriba, ¿en qué es diferente a la obtenida a RNN? ¿Es esto mejor o peor? ¿Por qué?

Las diferencias principales son que en la LSTM el training sigue bajando y se estabiliza pero el validation es mas alto. Ademas la RNN tenia curvas mas suaves. En terminos de si es mejor o peor, en este caso depende. En general, la grafica RNN parecia mejor pero viendo la prediccion, LSTM acierta mejor la secuencia.

- ¿Por qué LSTM puede funcionar mejor con secuencias largas?

Esto se debe a que LSTM tiene una estructura especializada para manejar la dependencia temporal. Usa celdas de memoria que le permiten recordar la información por más tiempo. Además controla que información retiene o descarta.

Parte 3 - Red Neuronal LSTM con PyTorch

Ahora que ya hemos visto el funcionamiento paso a paso de tanto RNN tradicional como LSTM. Es momento de usar PyTorch. Para esta parte usaremos el mismo dataset generado al inicio. Así mismo, usaremos un ciclo de entrenamiento similar al que hemos usado previamente.

En la siguiente parte (sí, hay una siguiente parte 🤖) usaremos otro tipo de dataset más formal

In [164...

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()

        # Aprox 1-3 lineas de codigo para declarar una capa LSTM
        self.lstm = nn.LSTM(input_size=vocab_size,
                             hidden_size=50,
                             num_layers=1,
                             bidirectional=False,
                             batch_first=False)

        # Hint: Esta tiene que tener el input_size del tamaño del vocabulario,
        # debe tener 50 hidden states (hidden_size)
        # una layer
        # y NO (False) debe ser bidireccional
        # YOUR CODE HERE
        # raise NotImplementedError()

        # Layer de salida (output)
        self.l_out = nn.Linear(in_features=50,
                                out_features=vocab_size,
                                bias=False)

    def forward(self, x):
        # RNN regresa el output y el ultimo hidden state
        x, (h, c) = self.lstm(x)

        # Aplanar la salida para una layer feed forward
        x = x.view(-1, self.lstm.hidden_size)

        # layer de output
        x = self.l_out(x)

        return x

net = Net()
print(net)
```

```

Net(
  (lstm): LSTM(4, 50)
  (l_out): Linear(in_features=50, out_features=4, bias=False)
)

```

```

In [165... # Hyper parametros
num_epochs = 500

# Init una nueva red
net = Net()

# Aprox 2 lineas para definir la función de perdida y el optimizador
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(net.parameters(), lr=1e-3)
# YOUR CODE HERE
# raise NotImplementedError()

# Perdida
training_loss, validation_loss = [], []

# Iteramos cada epoca
for i in range(num_epochs):

    # Perdidas
    epoch_training_loss = 0
    epoch_validation_loss = 0

    # NOTA 1
    net.eval()

    # Para cada secuencia en el validation set
    for inputs, targets in validation_set:

        # One-hot encode el inpyt y el target
        inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)
        targets_idx = [word_to_idx[word] for word in targets]

        # Convertir el input a un tensor
        inputs_one_hot = torch.Tensor(inputs_one_hot)
        inputs_one_hot = inputs_one_hot.permute(0, 2, 1)

        # Convertir el target a un tensor
        targets_idx = torch.LongTensor(targets_idx)

        # Aprox 1 linea para el Forward
        outputs = net(inputs_one_hot)
        # YOUR CODE HERE
        # raise NotImplementedError()

        # Aprox 1 linea para calcular la perdida
        loss = criterion(outputs, targets_idx)
        # Hint: Use el criterion definido arriba
        # YOUR CODE HERE
        # raise NotImplementedError()

    # Actualizacion de la perdida

```

```

        epoch_validation_loss += loss.detach().numpy()

# NOTA 2
net.train()

# Para cada secuencia en el training set
for inputs, targets in training_set:

    # One-hot encode el input y el target
    inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)
    targets_idx = [word_to_idx[word] for word in targets]

    # Convertir el input a un tensor
    inputs_one_hot = torch.Tensor(inputs_one_hot)
    inputs_one_hot = inputs_one_hot.permute(0, 2, 1)

    # Convertir el target a un tensor
    targets_idx = torch.LongTensor(targets_idx)

    # Aprox 1 linea para el Forward
    outputs = net(inputs_one_hot)
    # YOUR CODE HERE
    # raise NotImplementedError()

    # Aprox 1 linea para calcular la perdida
    loss = criterion(outputs, targets_idx)
    # Hint: Use el criterion definido arriba
    # YOUR CODE HERE
    # raise NotImplementedError()

    # Aprox 3 lineas para definir el backward
    optimizer.zero_grad() # Limpiar los gradientes
    loss.backward()
    optimizer.step()
    # YOUR CODE HERE
    # raise NotImplementedError()

    # Actualizacion de la perdida
    epoch_training_loss += loss.detach().numpy()

# Guardar la perdida para ser graficada
training_loss.append(epoch_training_loss/len(training_set))
validation_loss.append(epoch_validation_loss/len(validation_set))

# Mostrar la perdida cada 5 epocas
if i % 10 == 0:
    print(f'Epoch {i}, training loss: {training_loss[-1]}, validation loss: {va

```

```

Epoch 0, training loss: 1.1079386353492737, validation loss: 1.3830355048179626
Epoch 10, training loss: 0.3735764747485518, validation loss: 0.344302199780941
Epoch 20, training loss: 0.313075415417552, validation loss: 0.2782128691673279
Epoch 30, training loss: 0.3008677031844854, validation loss: 0.2722559362649918
Epoch 40, training loss: 0.30842086765915155, validation loss: 0.3506557047367096
Epoch 50, training loss: 0.2959584631025791, validation loss: 0.27338629961013794
Epoch 60, training loss: 0.29529520086944105, validation loss: 0.2734538972377777
Epoch 70, training loss: 0.29485927652567623, validation loss: 0.2734724894165993
Epoch 80, training loss: 0.29437013790011407, validation loss: 0.2734023749828339
Epoch 90, training loss: 0.2936120619997382, validation loss: 0.27356328815221786
Epoch 100, training loss: 0.29338778685778377, validation loss: 0.2736449122428894
Epoch 110, training loss: 0.2932410350069404, validation loss: 0.2734943240880966
Epoch 120, training loss: 0.2930354706943035, validation loss: 0.2733420133590698
Epoch 130, training loss: 0.2927589250728488, validation loss: 0.2731899946928024
Epoch 140, training loss: 0.2924421736970544, validation loss: 0.2730435386300087
Epoch 150, training loss: 0.292116466537118, validation loss: 0.2729014873504639
Epoch 160, training loss: 0.2917970307171345, validation loss: 0.27277960181236266
Epoch 170, training loss: 0.29148765057325365, validation loss: 0.27268157452344893
Epoch 180, training loss: 0.2911882931366563, validation loss: 0.27261477410793306
Epoch 190, training loss: 0.29089790768921375, validation loss: 0.2725829601287842
Epoch 200, training loss: 0.29062557853758336, validation loss: 0.2725591376423836
Epoch 210, training loss: 0.290374050475657, validation loss: 0.27250338196754453
Epoch 220, training loss: 0.2901335643604398, validation loss: 0.2723410800099373
Epoch 230, training loss: 0.2898862149566412, validation loss: 0.27207646071910857
Epoch 240, training loss: 0.28962888810783627, validation loss: 0.27200046479701995
Epoch 250, training loss: 0.28944887947291137, validation loss: 0.27255418300628664
Epoch 260, training loss: 0.28927479293197395, validation loss: 0.2727298840880394
Epoch 270, training loss: 0.2891187587752938, validation loss: 0.2728847235441208
Epoch 280, training loss: 0.2889796497300267, validation loss: 0.2730289131402969
Epoch 290, training loss: 0.28885470181703565, validation loss: 0.2731528177857399
Epoch 300, training loss: 0.28874224442988633, validation loss: 0.27325749695301055
Epoch 310, training loss: 0.28864111863076686, validation loss: 0.2733418166637421
Epoch 320, training loss: 0.28855135217308997, validation loss: 0.273402601480484
Epoch 330, training loss: 0.28846326638013126, validation loss: 0.2768136963248253
Epoch 340, training loss: 0.2884937455877662, validation loss: 0.27647709399461745
Epoch 350, training loss: 0.28844654839485884, validation loss: 0.2756820172071457
Epoch 360, training loss: 0.2883065206930041, validation loss: 0.2737478733062744
Epoch 370, training loss: 0.28830916006118057, validation loss: 0.27358560264110565
Epoch 380, training loss: 0.28827270716428754, validation loss: 0.27353044748306277
Epoch 390, training loss: 0.2882489785552025, validation loss: 0.27333829998970033
Epoch 400, training loss: 0.28818795960396526, validation loss: 0.27326942831277845
Epoch 410, training loss: 0.2881210124120116, validation loss: 0.2732659995555878
Epoch 420, training loss: 0.2880409676581621, validation loss: 0.27329278737306595
Epoch 430, training loss: 0.2879497217014432, validation loss: 0.2733150884509087
Epoch 440, training loss: 0.2878884023055434, validation loss: 0.27323615849018096
Epoch 450, training loss: 0.2878264898434281, validation loss: 0.2731139287352562
Epoch 460, training loss: 0.28774228133261204, validation loss: 0.27286951392889025
Epoch 470, training loss: 0.28766383063048123, validation loss: 0.27246857434511185
Epoch 480, training loss: 0.2876242157071829, validation loss: 0.2722498640418053
Epoch 490, training loss: 0.2875911548733711, validation loss: 0.27208456993103025

```

```

In [166... with tick.marks(5):
            assert compare_numbers(new_representation(training_loss[-1]), "3c3d", '0x1.28f5

with tick.marks(5):
            assert compare_numbers(new_representation(validation_loss[-1]), "3c3d", '0x1.28

```

✓ [5 marks]

✓ [5 marks]

In [167...

```
# Obtener la primera secuencia del test set
inputs, targets = test_set[1]

# One-hot encode el input y el target
inputs_one_hot = one_hot_encode_sequence(inputs, vocab_size)
targets_idx = [word_to_idx[word] for word in targets]

# Convertir el input a un tensor
inputs_one_hot = torch.Tensor(inputs_one_hot)
inputs_one_hot = inputs_one_hot.permute(0, 2, 1)

# Convertir el target a un tensor
targets_idx = torch.LongTensor(targets_idx)

# Aprox 1 linea para el Forward
outputs = net(inputs_one_hot)
# YOUR CODE HERE
# raise NotImplementedError()

print("Secuencia Input:")
print(inputs)

print("Secuencia Target:")
print(targets)

print("Secuencia Predicha:")
print([idx_to_word[np.argmax(output.detach().numpy())] for output in outputs])

# Graficar la perdida en training y validacion
epoch = np.arange(len(training_loss))
plt.figure()
plt.plot(epoch, training_loss, 'r', label='Training loss',)
plt.plot(epoch, validation_loss, 'b', label='Validation loss')
plt.legend()
plt.xlabel('Epoch'), plt.ylabel('NLL')
plt.show()
```


Secuencia Input:

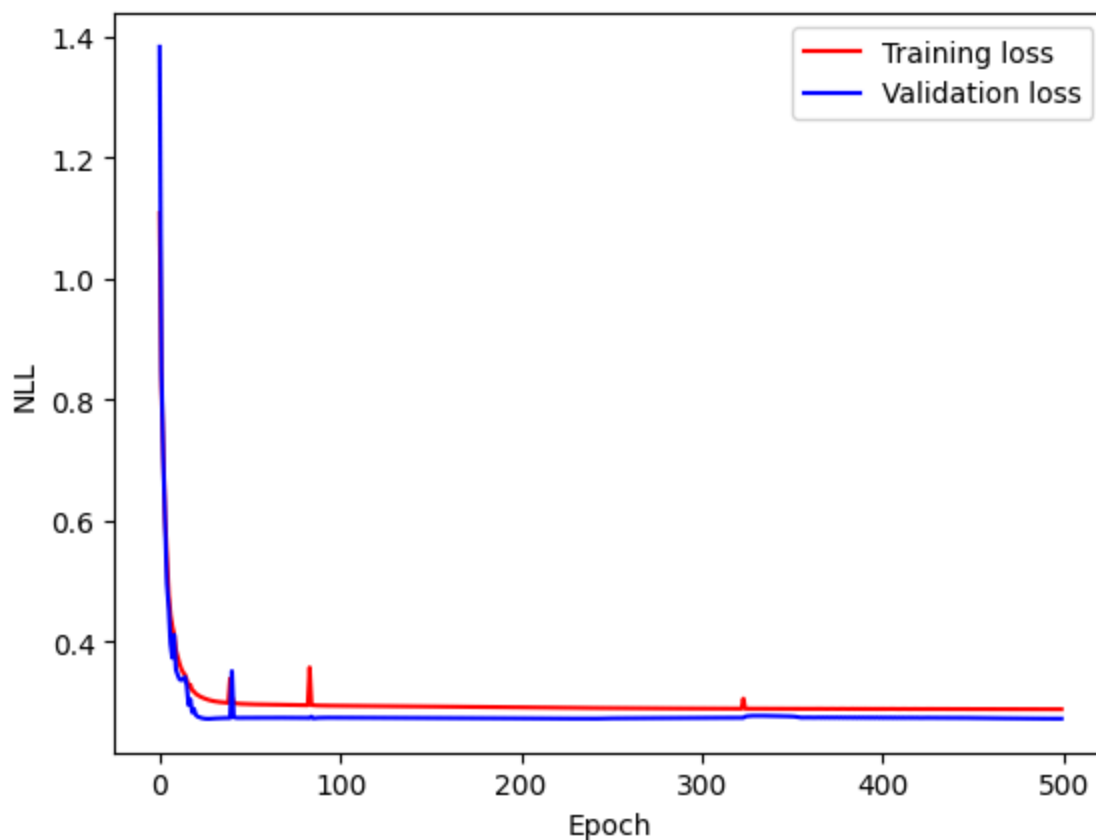
['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b']

Secuencia Target:

['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'EOS']

Secuencia Predicha:

['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'EOS']



Preguntas

Responda lo siguiente dentro de esta celda

- Compare las graficas obtenidas en el LSTM "a mano" y el LSTM "usando PyTorch, ¿cuál cree que es mejor? ¿Por qué?

Vemos que el LSTM con PyTorch es mejor por que la gráfica muestra menos perdida con unas curvas mas suaves y estables. Además vemos como es que PyTorch maneja mejor la optimización, los gradientes y la precisión numerica.

- Compare la secuencia target y la predicha de esta parte, ¿en qué parte falló el modelo?

Este modelo tuvo un pequeño fallo en la transición de "a" a "b", pero aunque tuvo estos pequeños fallos, el modelo capto bastante bien el patron.

- ¿Qué sucede en el código donde se señala "NOTA 1" y "NOTA 2"? ¿Para qué son necesarias estas líneas?

Estos cambian el modelo a modo evaluación para que se desactiven cosas como el dropout o el batchnorm si es que las hubiera. En la segunda vuelve a poner el modelo en modo entrenamiento para que se reactiven los componentes como el dropout

Parte 4 - Segunda Red Neuronal LSTM con PyTorch

Para esta parte será un poco menos guiada, por lo que se espera que puedan generar un modelo de Red Neuronal con LSTM para solventar un problema simple. Lo que se evaluará es la métrica final, y solamente se dejarán las generalidades de la implementación. El objetivo de esta parte, es dejar que ustedes exploren e investiguen un poco más por su cuenta.

En esta parte haremos uso de las redes LSTM pero para predicción de series de tiempo. Entonces lo que se busca es que dado un mes y un año, se debe predecir el número de pasajeros en unidades de miles. Los datos a usar son de 1949 a 1960.

Basado del blog "LSTM for Time Series Prediction in PyTorch" de Adrian Tam.

```
In [168... # Seed all
import torch
import random
import numpy as np

random.seed(seed_)
np.random.seed(seed_)
torch.manual_seed(seed_)
if torch.cuda.is_available():
    torch.cuda.manual_seed(seed_)
    torch.cuda.manual_seed_all(seed_) # Multi-GPU.
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

```
In [169... import pandas as pd

url_data = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passenger.csv"
dataset = pd.read_csv(url_data)
dataset.head(10)
```

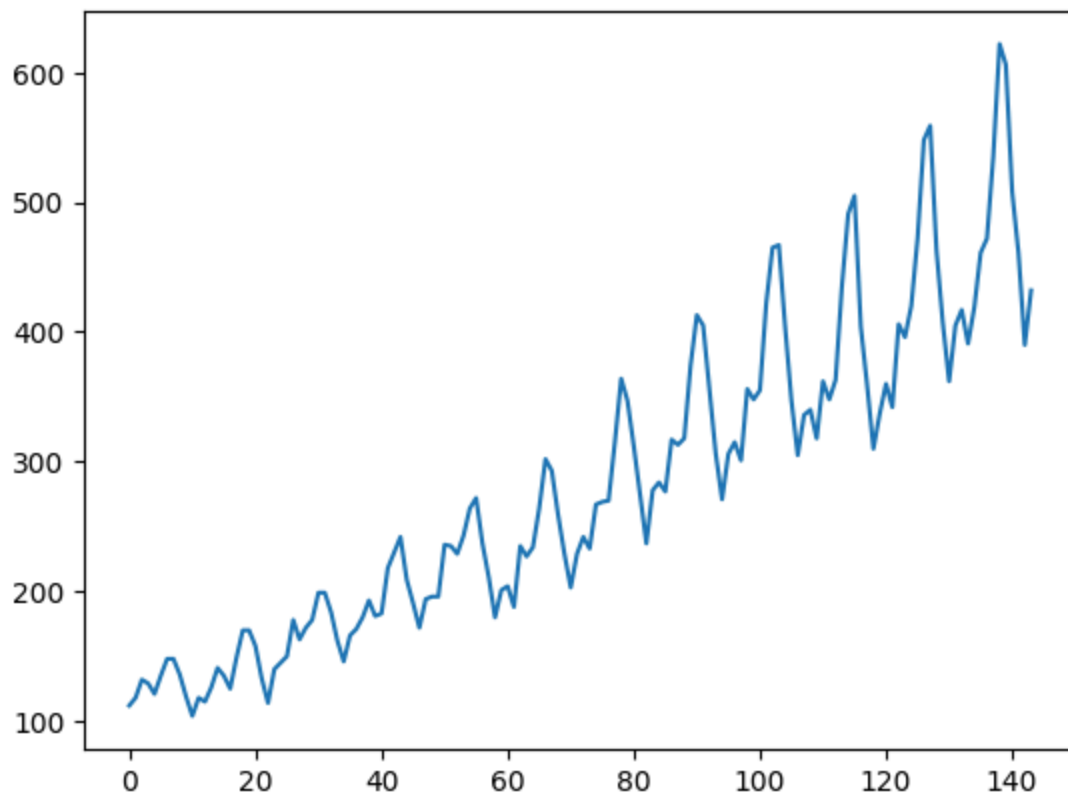
Out[169...

	Month	Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121
5	1949-06	135
6	1949-07	148
7	1949-08	148
8	1949-09	136
9	1949-10	119

In [170...

```
# Dibujemos la serie de tiempo
time_series = dataset[["Passengers"]].values.astype('float32')

plt.plot(time_series)
plt.show()
```



Esta serie de tiempo comprende 144 pasos de tiempo. El gráfico indica claramente una tendencia al alza y hay patrones periódicos en los datos que corresponden al período de vacaciones de verano. Por lo general, se recomienda "eliminar la tendencia" de la serie

temporal eliminando el componente de tendencia lineal y normalizándolo antes de continuar con el procesamiento. Sin embargo, por simplicidad de este ejercicios, vamos a omitir estos pasos.

Ahora necesitamos dividir nuestro dataset en training, validation y test set. A diferencia de otro tipo de datasets, cuando se trabaja en este tipo de proyectos, la división se debe hacer sin "revolver" los datos. Para esto, podemos hacerlo con NumPy

```
In [171... # En esta ocasion solo usaremos train y test, validation lo omitiremos para simplez  
# NO CAMBIEN NADA DE ESTA CELDA POR FAVOR  
p_train=0.8  
p_test=0.2  
  
# Definimos el tamaño de Las particiones  
num_train = int(len(time_series)*p_train)  
num_test = int(len(time_series)*p_test)  
  
# Dividir las secuencias en las particiones  
train = time_series[:num_train]  
test = time_series[num_train:]
```

El aspecto más complicado es determinar el método por el cual la red debe predecir la serie temporal. Por lo general, la predicción de series temporales se realiza en función de una ventana. En otras palabras, recibe datos del tiempo t_1 al t_2 , y su tarea es predecir para el tiempo t_3 (o más adelante). El tamaño de la ventana, denotado por w , dicta cuántos datos puede considerar el modelo al hacer la predicción. Este parámetro también se conoce como **look back period** (período retrospectivo).

Entonces, creemos una función para obtener estos datos, dado un look back period. Además, debemos asegurarnos de transformar estos datos a tensores para poder ser usados con PyTorch.

Esta función está diseñada para crear ventanas en la serie de tiempo mientras predice un paso de tiempo en el futuro inmediato. Su propósito es convertir una serie de tiempo en un tensor con dimensiones (muestras de ventana, pasos de tiempo, características). Dada una serie de tiempo con t pasos de tiempo, puede producir aproximadamente $(t - \text{ventana} + 1)$ ventanas, donde "ventana" denota el tamaño de cada ventana. Estas ventanas pueden comenzar desde cualquier paso de tiempo dentro de la serie de tiempo, siempre que no se extiendan más allá de sus límites.

Cada ventana contiene múltiples pasos de tiempo consecutivos con sus valores correspondientes, y cada paso de tiempo puede tener múltiples características. Sin embargo, en este conjunto de datos específico, solo hay una función disponible.

La elección del diseño garantiza que tanto la "característica" como el "objetivo" tengan la misma forma. Por ejemplo, para una ventana de tres pasos de tiempo, la "característica" corresponde a la serie de tiempo de $t-3$ a $t-1$, y el "objetivo" cubre los pasos de tiempo de $t-$

2 a t . Aunque estamos principalmente interesados en predecir $t+1$, la información de $t-2$ a t es valiosa durante el entrenamiento.

Es importante tener en cuenta que la serie temporal de entrada se representa como una matriz 2D, mientras que la salida de la función `create_timeseries_dataset()` será un tensor 3D. Para demostrarlo, usemos `lookback=1` y verifiquemos la forma del tensor de salida en consecuencia.

In [172...

```
import torch

def create_timeseries_dataset(dataset, lookback):
    X, y = [], []
    for i in range(len(dataset) - lookback):
        feature = dataset[i : i + lookback]
        target = dataset[i + 1 : i + lookback + 1]
        X.append(feature)
        y.append(target)
    return torch.tensor(X), torch.tensor(y)

# EL VALOR DE LB SÍ LO PUEDEN CAMBIAR SI LO CONSIDERAN NECESARIO
lb = 4
X_train, y_train = create_timeseries_dataset(train, lookback=lb)
#X_validation, y_validation = create_timeseries_dataset(validation, lookback=lb)
X_test, y_test = create_timeseries_dataset(test, lookback=lb)

print(X_train.shape, y_train.shape)
#print(X_validation.shape, y_validation.shape)
print(X_test.shape, y_test.shape)
```

```
torch.Size([111, 4, 1]) torch.Size([111, 4, 1])
torch.Size([25, 4, 1]) torch.Size([25, 4, 1])
```

Ahora necesitamos crear una clase que definirá nuestro modelo de red neuronal con LSTM. Noten que acá solo se dejaron las firmas de las funciones necesarias, ustedes deberán decidir que arquitectura con LSTM implementar, con la finalidad de superar cierto threshold de métrica de desempeño mencionado abajo.

In [173...

```
import torch.nn as nn

# NOTA: Moví el numero de iteraciones para que no se borre al ser evaluado
# Pueden cambiar el número de épocas en esta ocasión con tal de llegar al valor de
n_epochs = 3000
# YOUR CODE HERE
# raise NotImplementedError()

class CustomModelLSTM(nn.Module):
    def __init__(self):
        super(CustomModelLSTM, self).__init__()

        self.lstm = nn.LSTM(
            input_size=1,
            hidden_size=50,
            num_layers=1,
```

```

        bidirectional=False,
        batch_first=False
    )

    self.l_out = nn.Linear(in_features=50, out_features=1, bias=False)

    def forward(self, x):
        x, (h, c) = self.lstm(x)
        x = x.view(-1, self.lstm.hidden_size)
        x = self.l_out(x)
        x = x.view(-1, 1, 1)
        return x

```

La función `nn.LSTM()` produce una tupla como salida. El primer elemento de esta tupla consiste en los hidden states generados, donde cada paso de tiempo de la entrada tiene su correspondiente hidden state. El segundo elemento contiene la memoria y los hidden states de la unidad LSTM, pero no se usan en este contexto particular.

La capa LSTM se configura con la opción `batch_first=True` porque los tensores de entrada se preparan en la dimensión de (muestra de ventana, pasos de tiempo, características). Con esta configuración, se crea un batch tomando muestras a lo largo de la primera dimensión.

Para generar un único resultado de regresión, la salida de los estados ocultos se procesa aún más utilizando una capa fully connected. Dado que la salida de LSTM corresponde a un valor para cada paso de tiempo de entrada, se debe seleccionar solo la salida del último paso de tiempo.

In [174...

```

import torch.optim as optim
import torch.utils.data as data

# NOTEN QUE ESTOY PONIENDO DE NUEVO LOS SEEDS PARA SER CONSTANTES
random.seed(seed_)
np.random.seed(seed_)
torch.manual_seed(seed_)
if torch.cuda.is_available():
    torch.cuda.manual_seed(seed_)
    torch.cuda.manual_seed_all(seed_) # Multi-GPU.
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
#####

model = CustomModelLSTM()
# Optimizador y perdida
optimizer = optim.Adam(model.parameters())
loss_fn = nn.MSELoss()
# Observen como podemos también definir un DataLoader de forma snecilla
loader = data.DataLoader(data.TensorDataset(X_train, y_train), shuffle=False, batch

# Perdidas
loss_train = []

```

```

loss_test = []

# Iteramos sobre cada epoca
for epoch in range(n_epochs):
    # Colocamos el modelo en modo de entrenamiento
    model.train()

    # Cargamos Los batches
    for X_batch, y_batch in loader:
        # Obtenemos una primera prediccion
        y_pred = model(X_batch)
        # Calculamos la perdida
        loss = loss_fn(y_pred, y_batch)
        # Reseteamos la gradiente a cero
        # sino la gradiente de previas iteraciones se acumulará con las nuevas
        optimizer.zero_grad()
        # Backprop
        loss.backward()
        # Aplicar las gradientes para actualizar los parametros del modelo
        optimizer.step()

    # Validación cada 100 epocas
    if epoch % 100 != 0 and epoch != n_epochs-1:
        continue
    # Colocamos el modelo en modo de evaluación
    model.eval()

    # Deshabilitamos el calculo de gradientes
    with torch.no_grad():
        # Prediccion
        y_pred = model(X_train)
        # Calculo del RMSE - Root Mean Square Error
        train_rmse = np.sqrt(loss_fn(y_pred, y_train))
        # Prediccion sobre validation
        y_pred = model(X_test)
        # Calculo del RMSE para validation
        test_rmse = np.sqrt(loss_fn(y_pred, y_test))
        loss_train.append(train_rmse)
        loss_test.append(test_rmse)

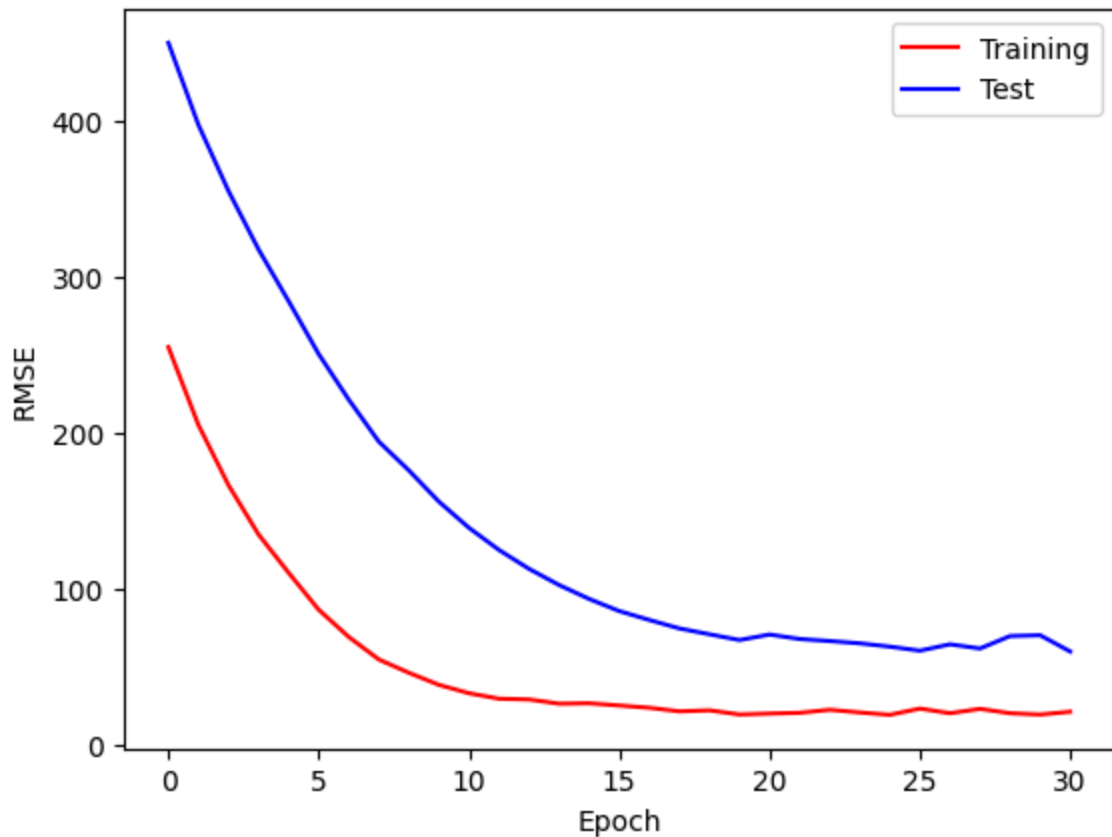
    print("Epoch %d: train RMSE %.4f, test RMSE %.4f" % (epoch, train_rmse, test_rm

```

```
Epoch 0: train RMSE 254.8259, test RMSE 449.6753
Epoch 100: train RMSE 204.9441, test RMSE 397.1295
Epoch 200: train RMSE 166.4205, test RMSE 354.7715
Epoch 300: train RMSE 134.8459, test RMSE 317.3653
Epoch 400: train RMSE 110.3341, test RMSE 284.2560
Epoch 500: train RMSE 86.6714, test RMSE 250.3943
Epoch 600: train RMSE 69.2341, test RMSE 221.2180
Epoch 700: train RMSE 54.8275, test RMSE 194.3416
Epoch 800: train RMSE 46.2929, test RMSE 175.8872
Epoch 900: train RMSE 38.5571, test RMSE 155.8820
Epoch 1000: train RMSE 33.2165, test RMSE 139.1157
Epoch 1100: train RMSE 29.6820, test RMSE 124.9381
Epoch 1200: train RMSE 29.2210, test RMSE 112.8183
Epoch 1300: train RMSE 26.4975, test RMSE 102.4633
Epoch 1400: train RMSE 26.7760, test RMSE 93.5582
Epoch 1500: train RMSE 25.3480, test RMSE 85.8067
Epoch 1600: train RMSE 23.8916, test RMSE 80.1215
Epoch 1700: train RMSE 21.5958, test RMSE 74.6883
Epoch 1800: train RMSE 22.2619, test RMSE 70.9494
Epoch 1900: train RMSE 19.5869, test RMSE 67.2592
Epoch 2000: train RMSE 20.1917, test RMSE 70.7999
Epoch 2100: train RMSE 20.6577, test RMSE 67.9352
Epoch 2200: train RMSE 22.6151, test RMSE 66.6269
Epoch 2300: train RMSE 20.8931, test RMSE 65.1587
Epoch 2400: train RMSE 19.3752, test RMSE 63.0480
Epoch 2500: train RMSE 23.3788, test RMSE 60.4577
Epoch 2600: train RMSE 20.3984, test RMSE 64.5369
Epoch 2700: train RMSE 23.2047, test RMSE 61.8540
Epoch 2800: train RMSE 20.3928, test RMSE 69.8147
Epoch 2900: train RMSE 19.5564, test RMSE 70.3486
Epoch 2999: train RMSE 21.3530, test RMSE 59.9076
```

In [175...

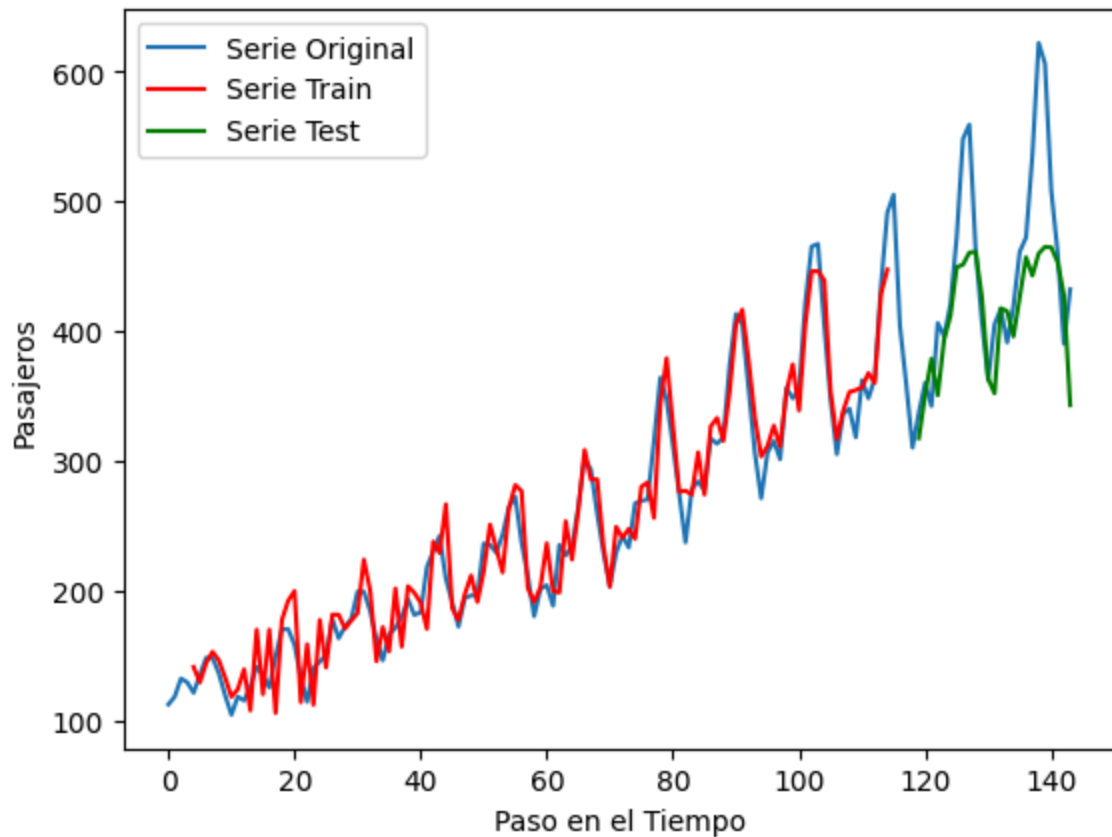
```
# Visualización del rendimiento
epoch = np.arange(len(loss_train))
plt.figure()
plt.plot(epoch, loss_train, 'r', label='Training',)
plt.plot(epoch, loss_test, 'b', label='Test')
plt.legend()
plt.xlabel('Epoch'), plt.ylabel('RMSE')
plt.show()
```

In [176...

```
# Graficamos
with torch.no_grad():
    # Movemos las predicciones de train para graficar
    train_plot = np.ones_like(time_series) * np.nan
    # Prediccion de train
    y_pred = model(X_train)
    # Extraemos los datos solo del ultimo paso
    y_pred = y_pred[:, -1, :]
    train_plot[lb : num_train] = model(X_train)[:, -1, :]
    # Movemos las predicciones de test
    test_plot = np.ones_like(time_series) * np.nan
    test_plot[num_train + lb : len(time_series)] = model(X_test)[:, -1, :]

plt.figure()
plt.plot(time_series, label="Serie Original")
plt.plot(train_plot, c='r', label="Serie Train")
plt.plot(test_plot, c='g', label="Serie Test")
plt.xlabel('Paso en el Tiempo'), plt.ylabel('Pasajeros')
plt.legend()
plt.show()
```



Nota: Lo que se estará evaluando es el RMSE tanto en training como en test. Se evaluará que en training sea **menor a 22**, mientras que en testing sea **menor a 70**.

```
In [177... float(loss_test[len(loss_test)-1])
float(test_rmse)
loss_train

with tick.marks(7):
    assert loss_train[-1] < 22

with tick.marks(7):
    assert train_rmse < 22

with tick.marks(7):
    assert loss_test[-1] < 70

with tick.marks(7):
    assert test_rmse < 70
```

✓ [7 marks]

✓ [7 marks]

✓ [7 marks]

✓ [7 marks]

In [178...

```
print()  
print("La fraccion de abajo muestra su rendimiento basado en las partes visibles de  
tick.summarise_marks() #
```

La fraccion de abajo muestra su rendimiento basado en las partes visibles de este laboratorio

153 / 153 marks (100.0%)

In []: